

Segment Descriptors

Xeno Kovah – 2021

xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- <http://creativecommons.org/licenses/by-sa/4.0/>

You are free:



to **Share** — to copy, distribute and transmit the work



to **Remix** — to adapt the work

Under the following conditions:



Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).



Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

GDT & LDT

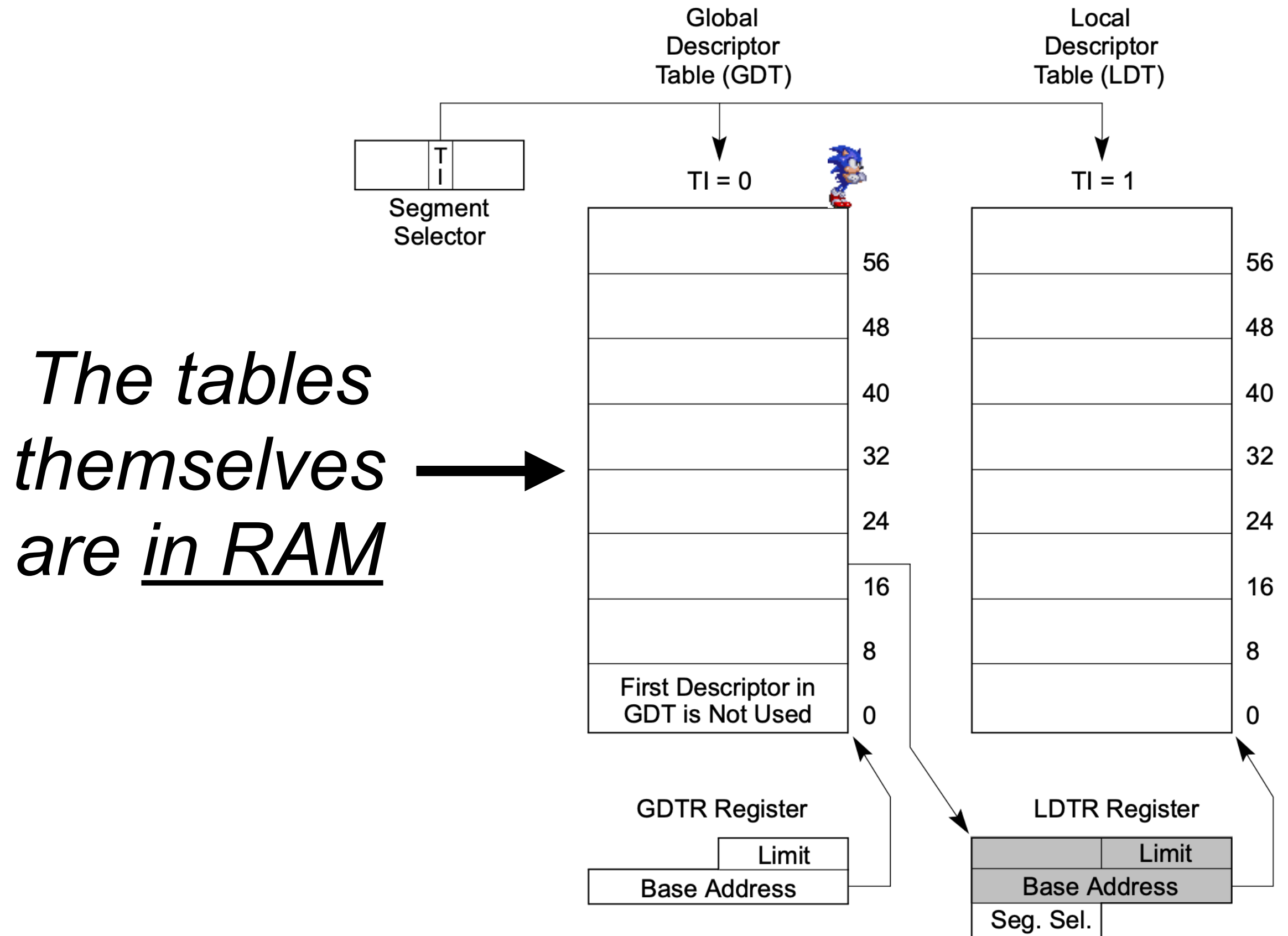


Figure 3-10. Global and Local Descriptor Tables

Segment Descriptors 1

- Each entry in the GDT and LDT is a data structure called a Segment Descriptor
- Each Segment Descriptor describes a segment :P

Segment Descriptors 2

- “Each segment has a segment descriptor, which specifies the size of the segment, the access rights and ***privilege level*** for the segment, the segment type, and the location of the first byte of the segment in the linear address space (called the base address of the segment).”

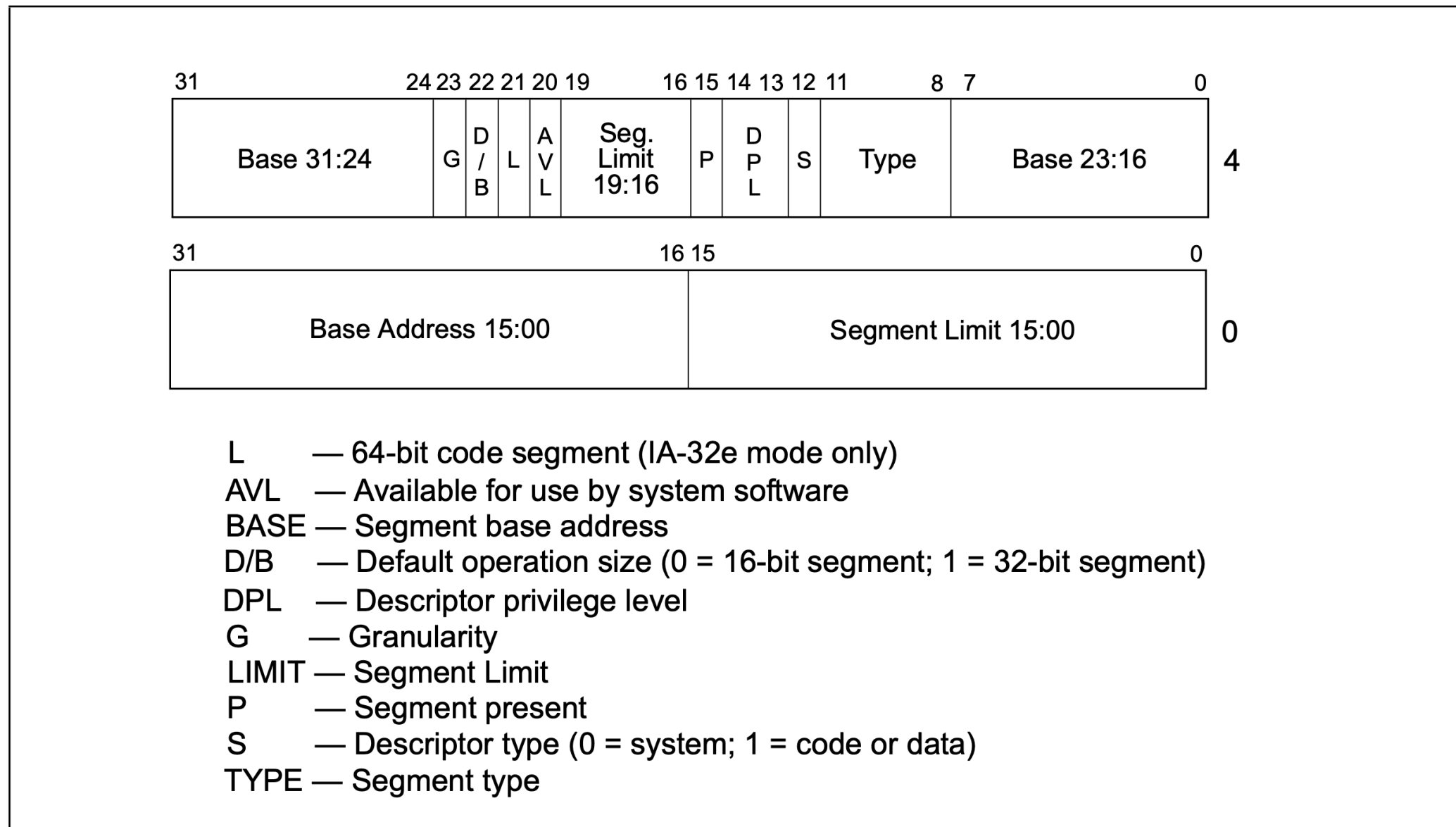


Figure 3-8. Segment Descriptor

Segment Descriptors 2

- “Each segment has a segment descriptor, which specifies the size of the segment, the access rights and ***privilege level*** for the segment, the segment type, and the location of the first byte of the segment in the linear address space (called the base address of the segment).”

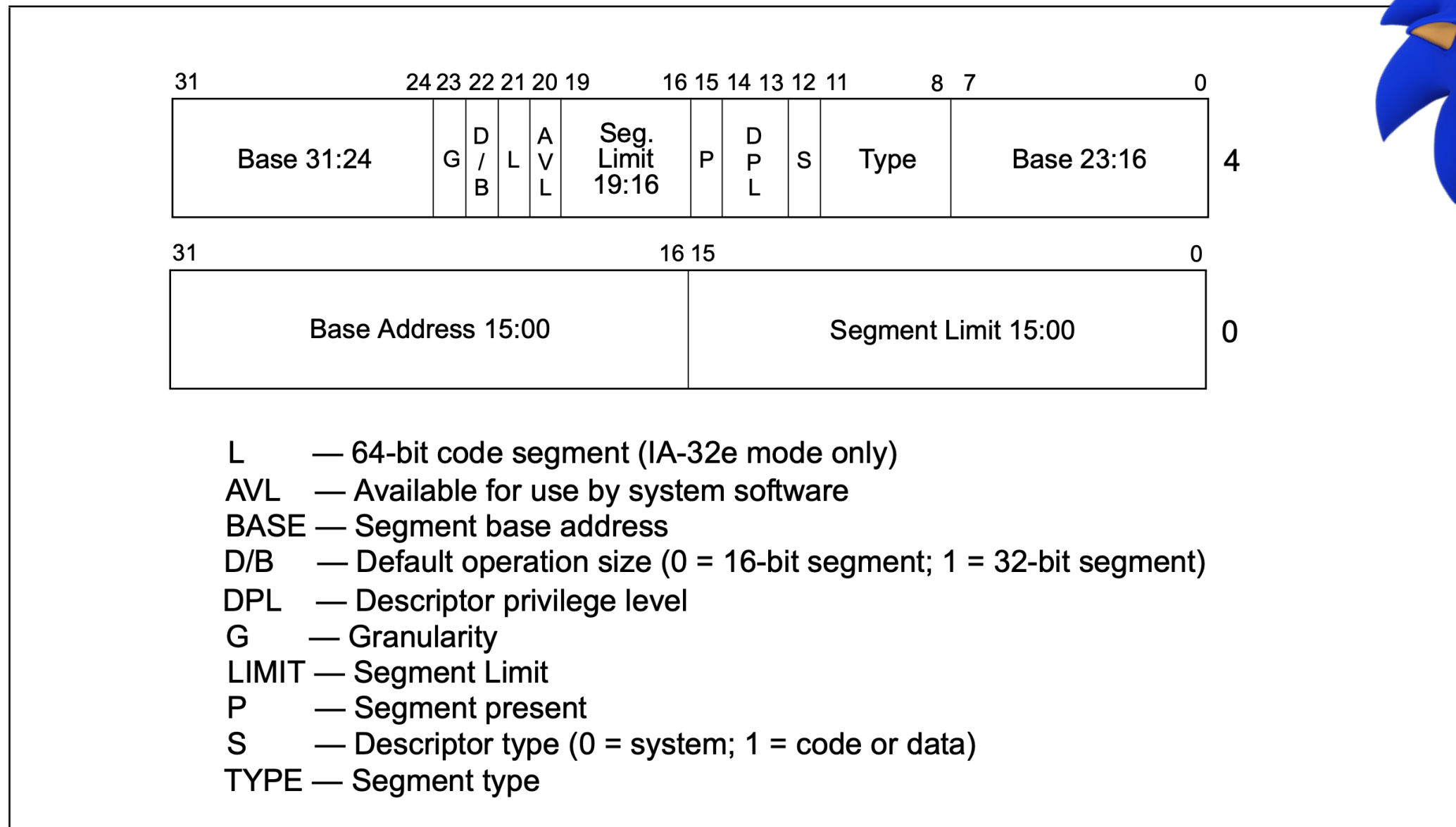
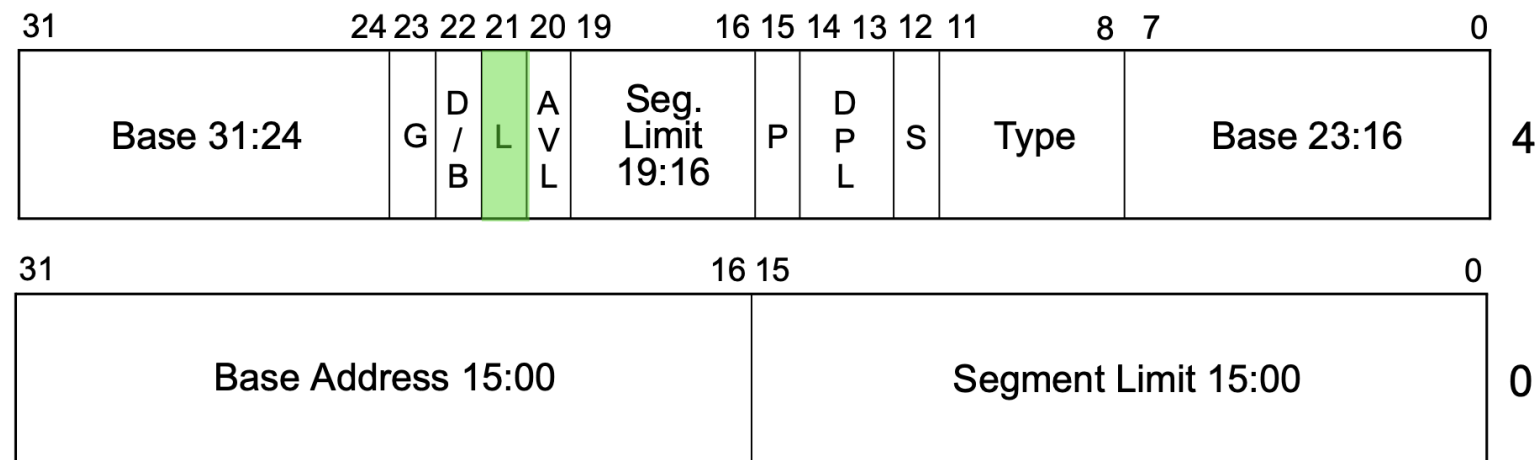
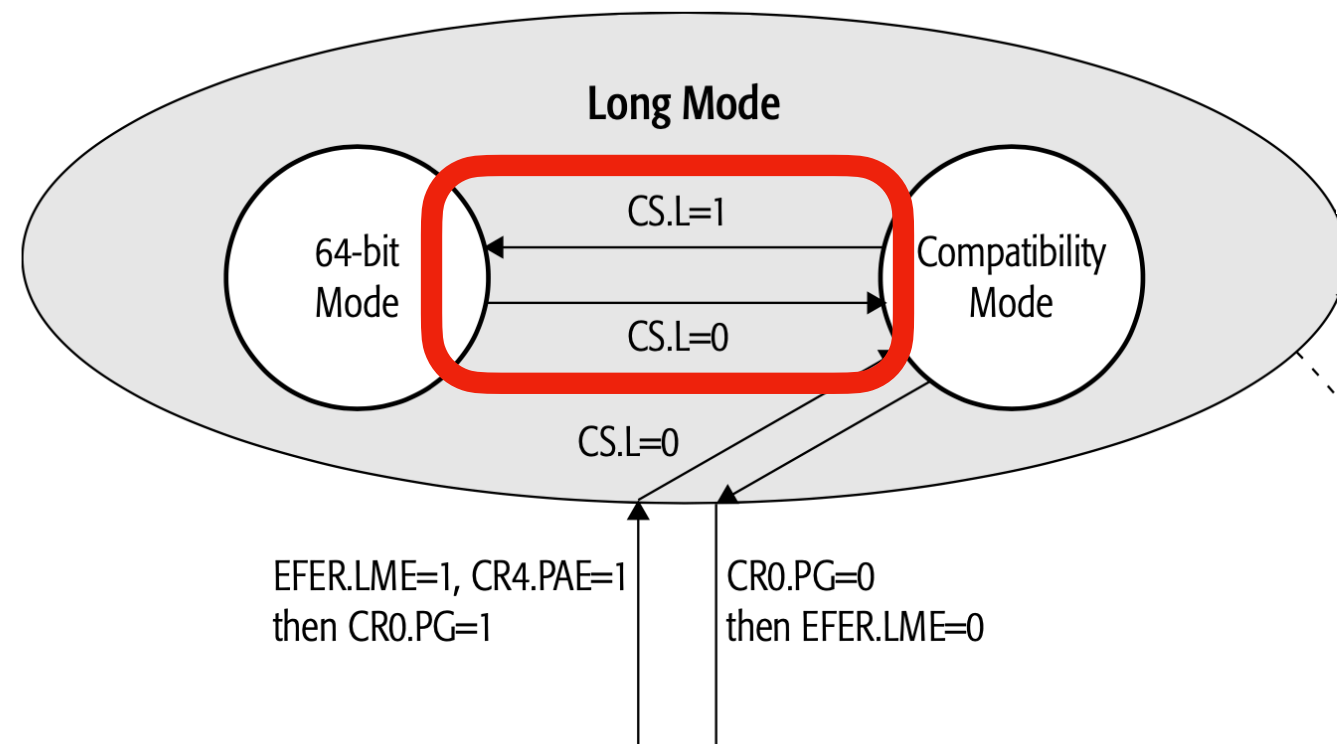


Figure 3-8. Segment Descriptor

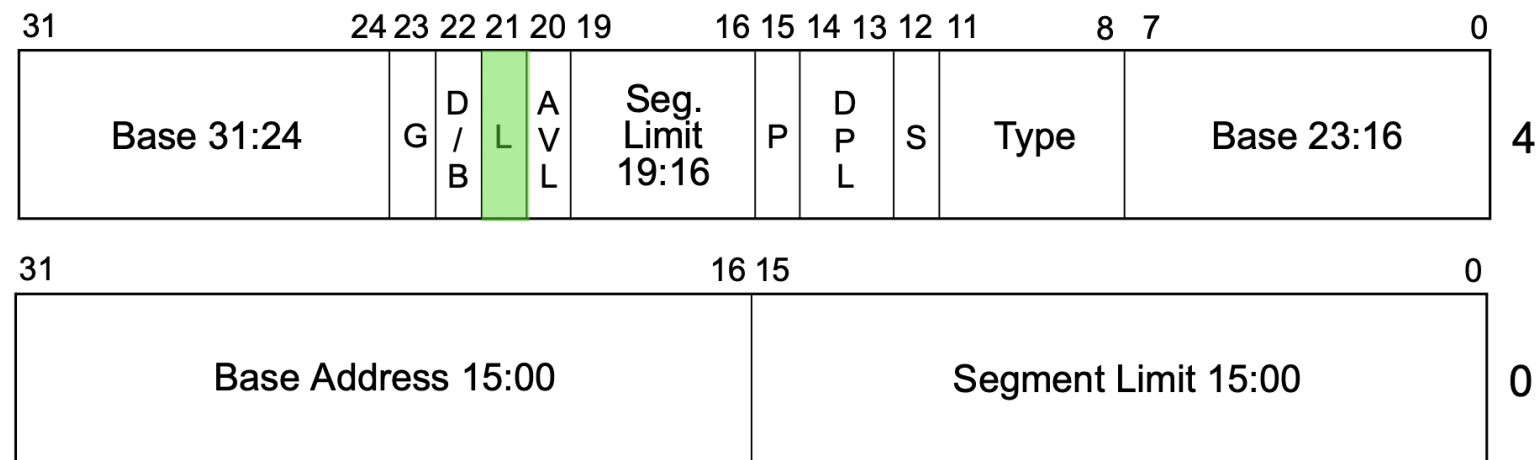
Long Mode Flag



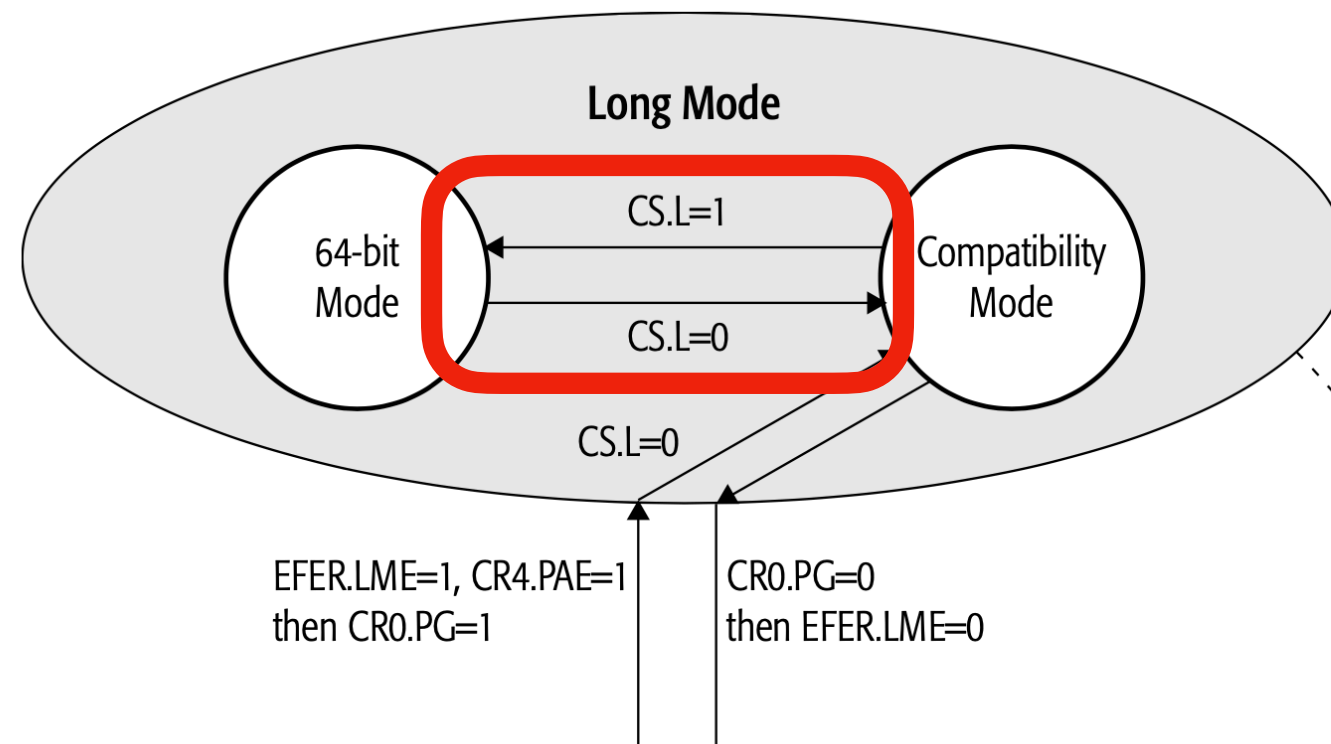
- L Flag (bit 21): Whether this is a 64-bit segment or not (if not, it's Compatibility Mode, which exhibits historical 32-bit protected mode behavior)



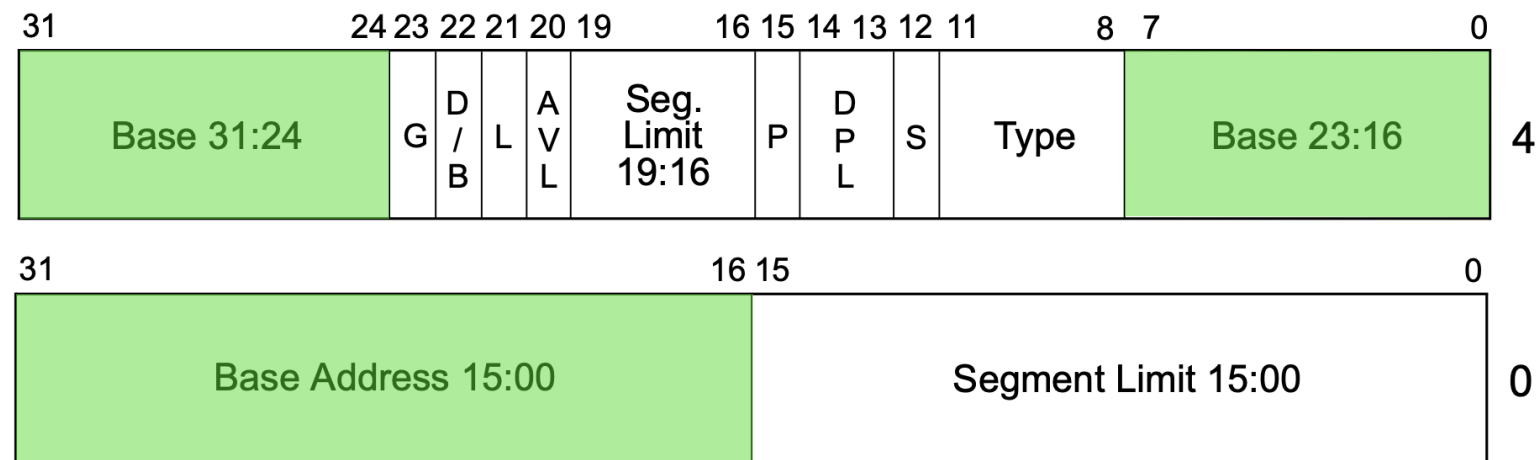
Long Mode Flag



- L Flag (bit 21): Whether this is a 64-bit segment or not (if not, it's Compatibility Mode, which exhibits historical 32-bit protected mode behavior)

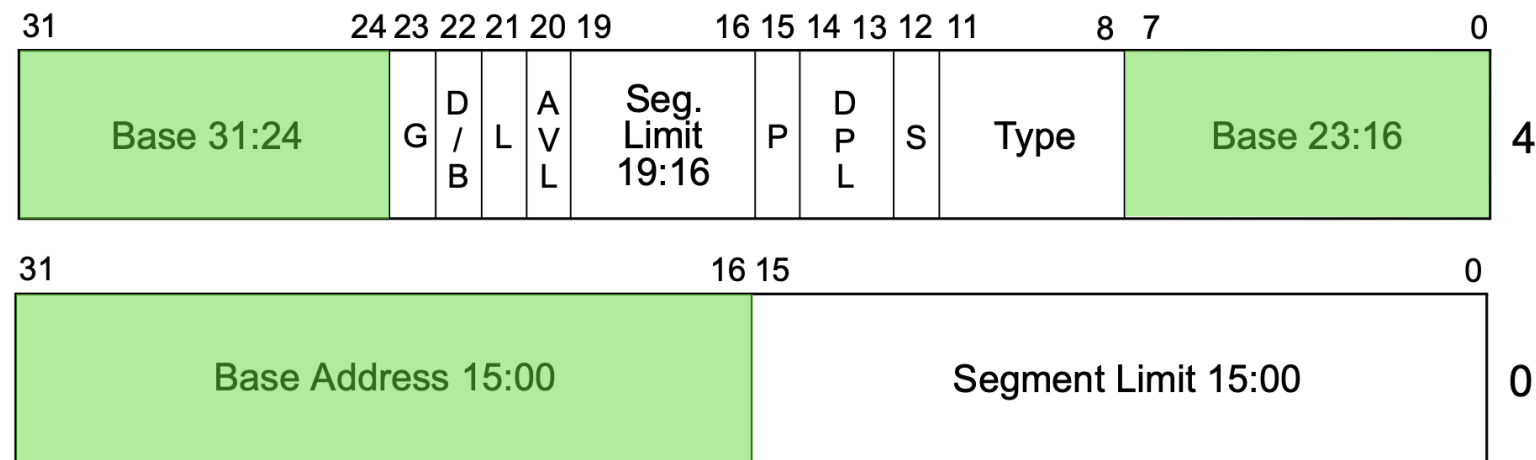


Segment Base



- Base (32 bits): linear address where the segment starts, *for Compatibility Mode*
- For 64-bit mode, the base is just always treated as 0 *for CS, SS, DS, ES*
- *For FS and GS, the hidden part of the segment registers is mapped into the IA32_FS_BASE (0xC0000100) and IA32_GS_BASE (0xC0000101) MSRs respectively. So a full 64 bit address can be written there instead of in the segment descriptor (Vol 3, Sect 3.4.4)*

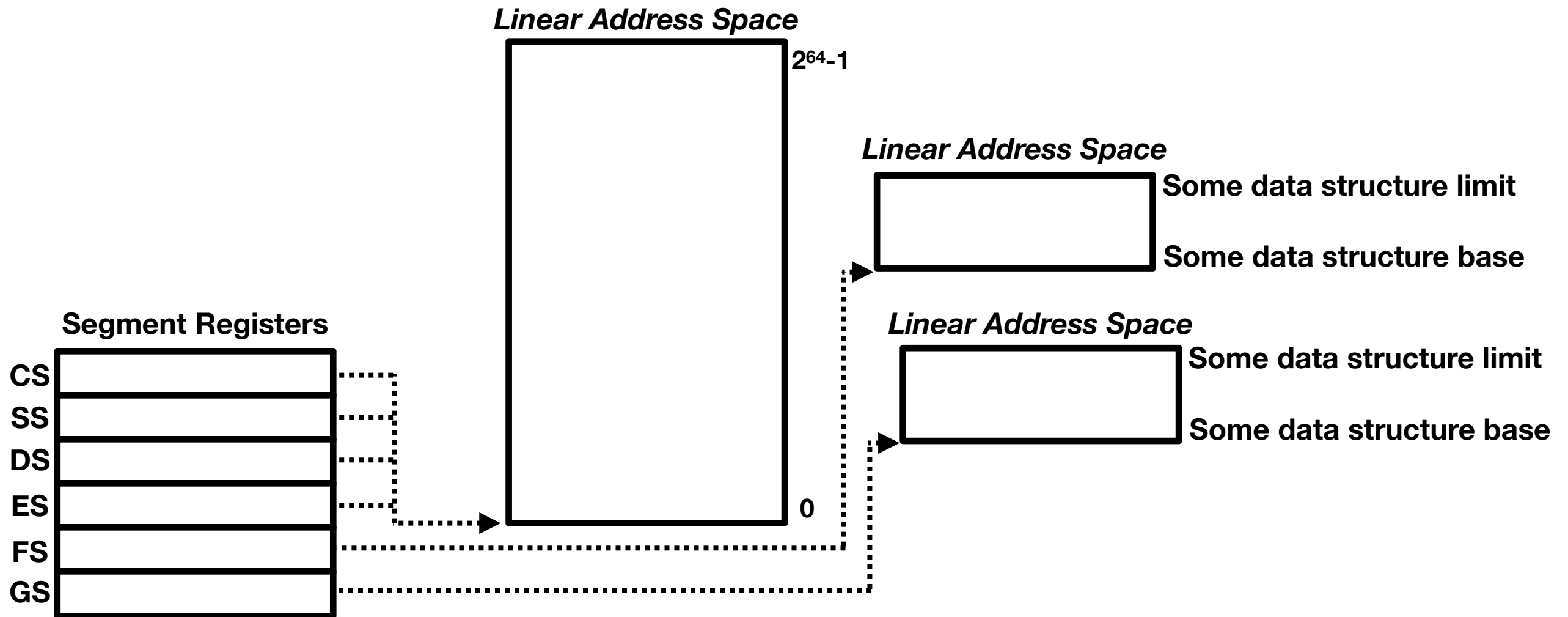
Segment Base



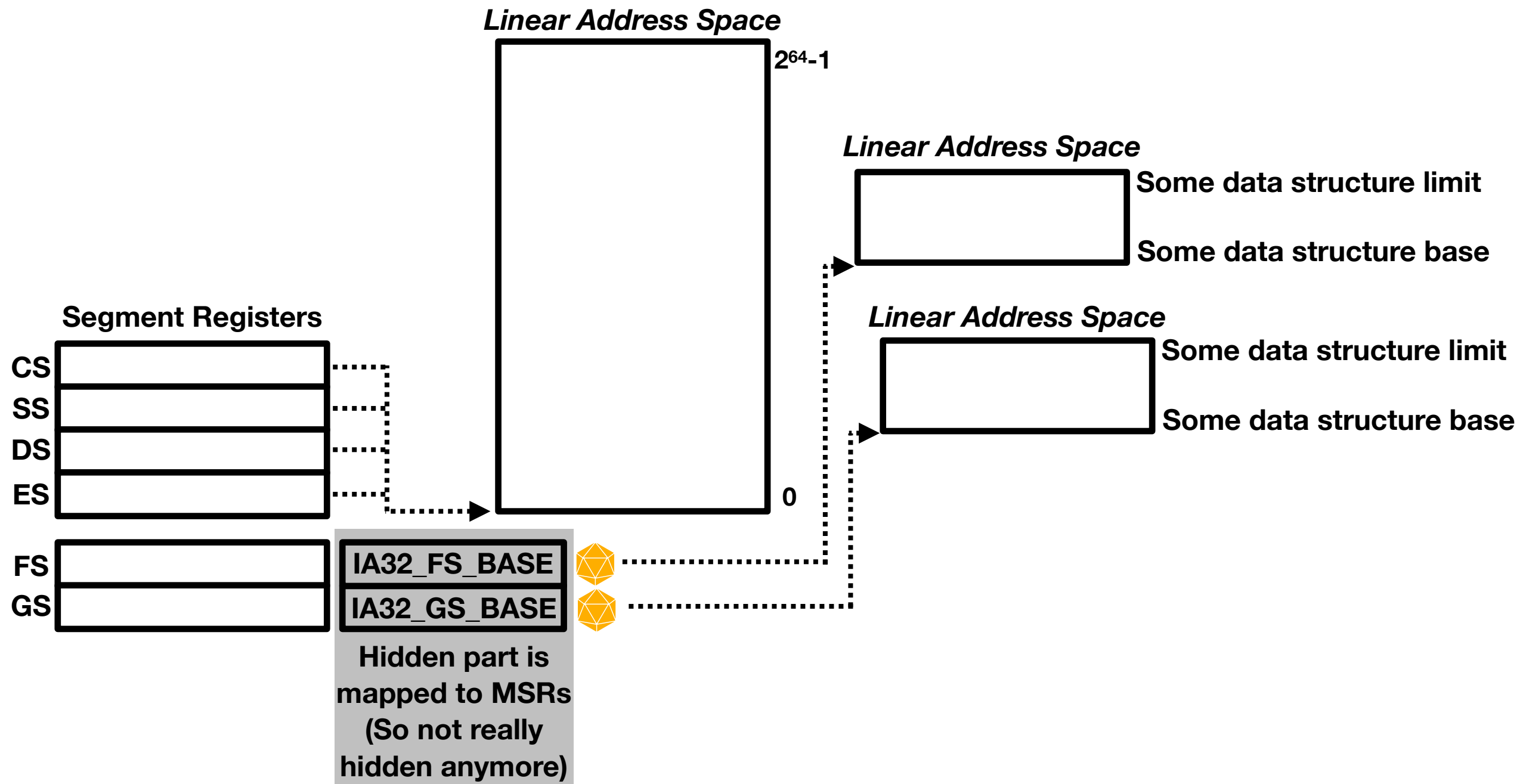
- Base (32 bits): linear address where the segment starts, *for Compatibility Mode*
- For 64-bit mode, the base is just always treated as 0 *for CS, SS, DS, ES*
- *For FS and GS, the hidden part of the segment registers is mapped into the IA32_FS_BASE (0xC0000100) and IA32_GS_BASE (0xC0000101) MSRs respectively. So a full 64 bit address can be written there instead of in the segment descriptor (Vol 3, Sect 3.4.4)*



We Saw This...



And Just Learned This...



Results for Win10 20H2 x86-64

(subject to change on other versions, service pack levels, etc)

U_SegRegs

Segment Register	RPL	Table	Index
CS = 0x33	3	GDT	6
SS = 0x2b	3	GDT	5
DS = 0x2b	3	GDT	5
ES = 0x2b	3	GDT	5
FS = 0x53	3	GDT	0xA
GS = 0x2b	3	GDT	5

K_SegRegs

Segment Register	RPL	Table	Index
CS = 0x10	0	GDT	2
SS = 0x18	0	GDT	3
DS = 0x2b	3	GDT	5
ES = 0x2b	3	GDT	5
FS = 0x53	3	GDT	0xA
GS = 0x2b	3	GDT	5

==?



Results for Win10 20H2 x86-64

(subject to change on other versions, service pack levels, etc)

U_SegRegs

Segment Register	RPL	Table	Index
CS = 0x33	3	GDT	6
SS = 0x2b	3	GDT	5
DS = 0x2b	3	GDT	5
ES = 0x2b	3	GDT	5
FS = 0x53	3	GDT	0xA
GS = 0x2b	3	GDT	5

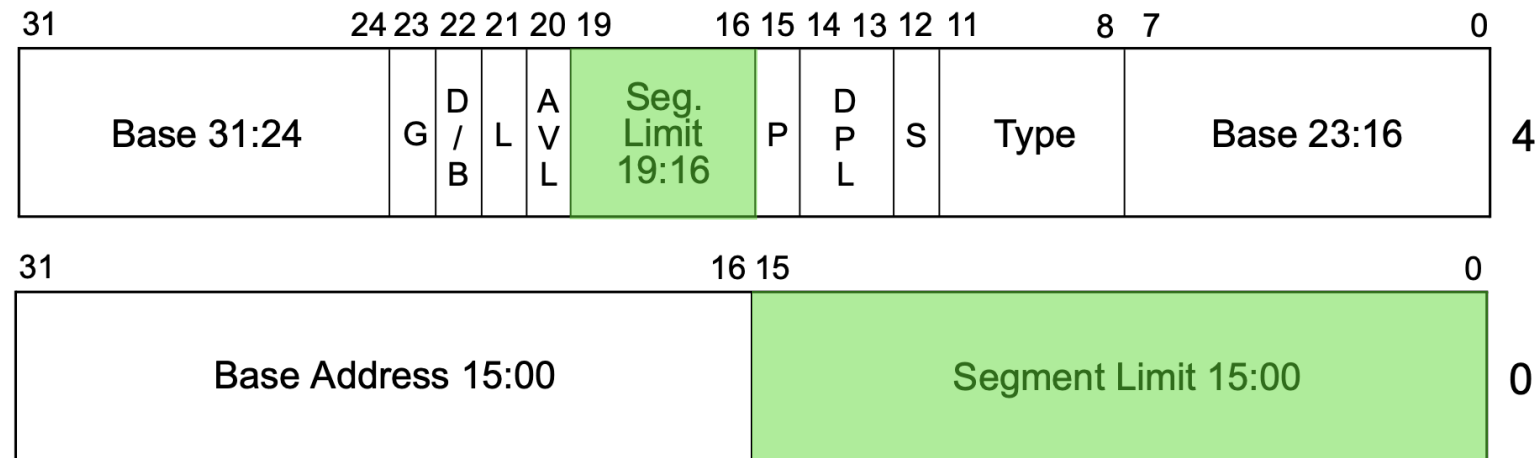
K_SegRegs

Segment Register	RPL	Table	Index
CS = 0x10	0	GDT	2
SS = 0x18	0	GDT	3
DS = 0x2b	3	GDT	5
ES = 0x2b	3	GDT	5
FS = 0x53	3	GDT	0xA
GS = 0x2b	3	GDT	5

==?

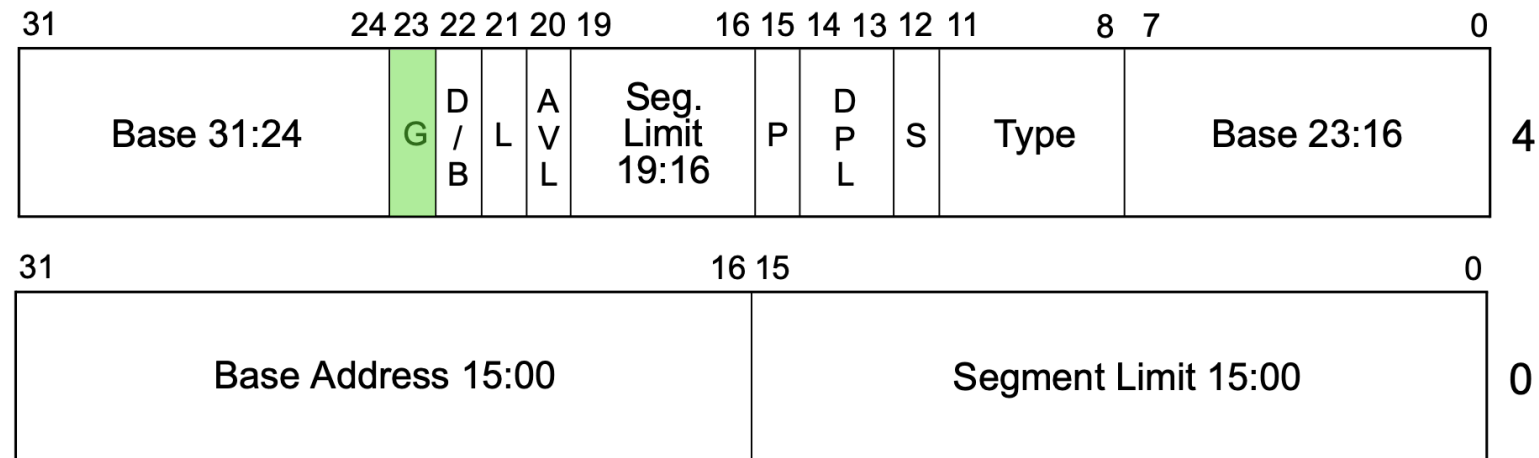


Segment Limit



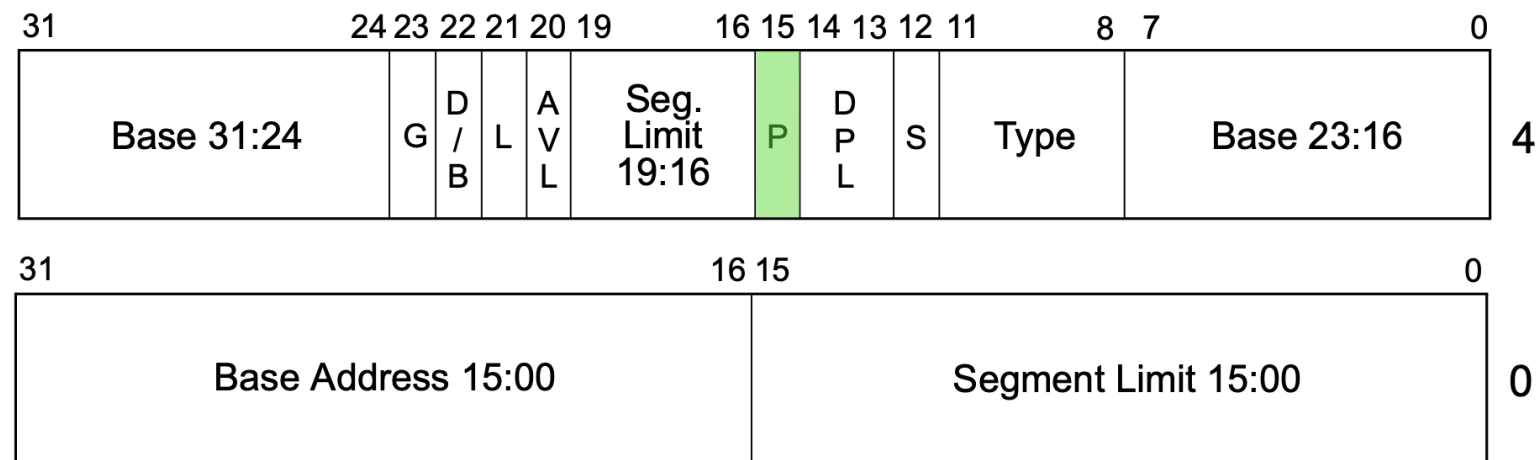
- Limit (20 bits): size of the segment (in bytes or 4kb blocks), *for Compatibility Mode*
- Limits are not actually checked anymore in 64 bit mode, even for FS and GS

Granularity Flag



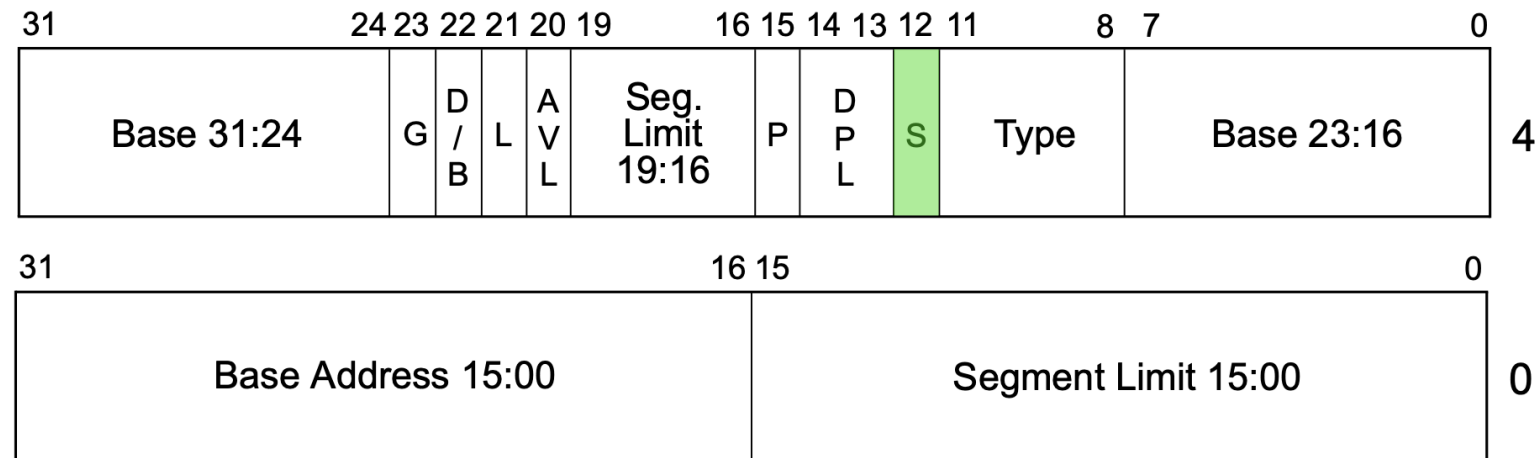
- G Flag: whether the Limit is specified in bytes or 4096 byte chunks, *for Compatibility Mode*
- Not used in 64-bit mode

Present Flag



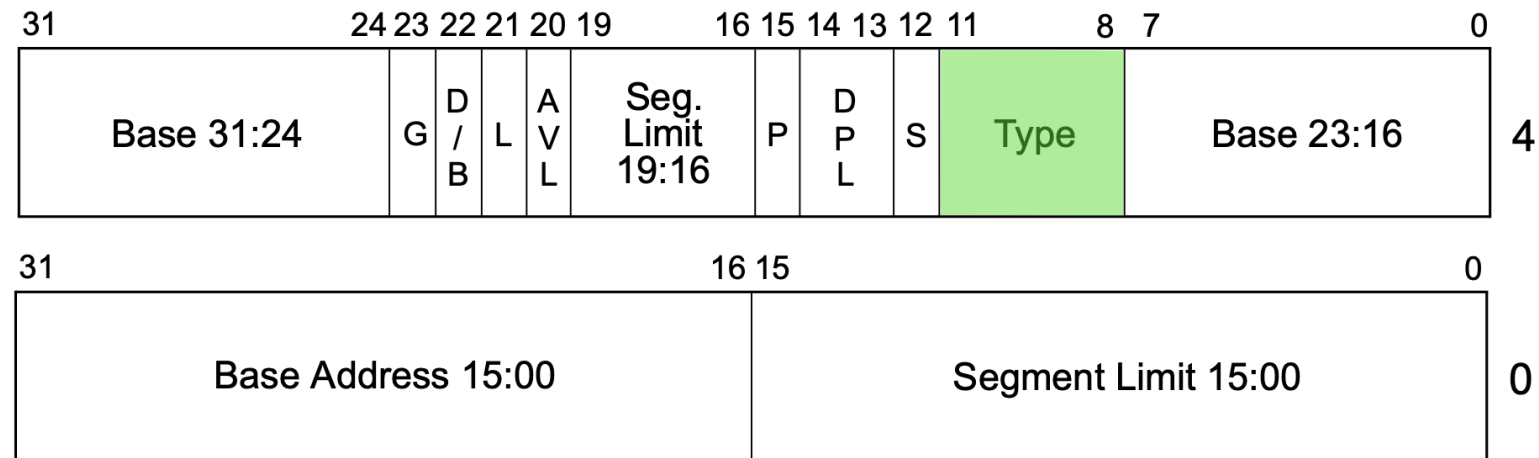
- P Flag: 0 for not present. 1 for present
 - “If this flag is clear, the processor generates a segment-not-present exception (#NP) when a segment selector that points to the segment descriptor is loaded into a segment register.”

System Flag



- S Flag: 0 for System segment! 1 for Code or Data segment (***kinda backwards!*** 😬)

Type Flags



- Type (4 bits): Different types for System and non-System (i.e. Code & Data) descriptors
- See next slides

Table 3-1. Code- and Data-Segment Types

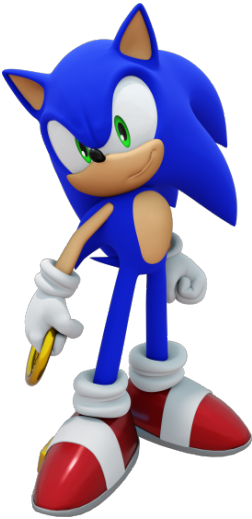
Type Field					Descriptor Type	Description
Decimal	11	10 E	9 W	8 A		
0	0	0	0	0	Data	Read-Only
1	0	0	0	1	Data	Read-Only, accessed
2	0	0	1	0	Data	Read/Write
3	0	0	1	1	Data	Read/Write, accessed
4	0	1🙄	0	0	Data	Read-Only, expand-down
5	0	1🙄	0	1	Data	Read-Only, expand-down, accessed
6	0	1🙄	1	0	Data	Read/Write, expand-down
7	0	1🙄	1	1	Data	Read/Write, expand-down, accessed
		C	R	A		
8	1	0🤪	0	0	Code	Execute-Only
9	1	0	0	1	Code	Execute-Only, accessed
10	1	0	1	0	Code	Execute/Read
11	1	0	1	1	Code	Execute/Read, accessed
12	1	1	0	0	Code	Execute-Only, conforming
13	1	1	0	1	Code	Execute-Only, conforming, accessed
14	1	1	1	0	Code	Execute/Read, conforming
15	1	1	1	1	Code	Execute/Read, conforming, accessed

What's the Point of Expand-Down?

- Originally for Stack Segments, to allow for growth towards lower addresses
- Although a read-only && expand-down segment can't be used for stack segments...
- "Loading the SS register with a segment selector for a nonwritable data segment generates a general-protection exception (#GP)." (Vol 3, Sect 3.4.5.1)

What's the Point of Conforming vs. Non-Conforming Segments? 1

- Conforming segments 🙄 allow lower privilege code to execute them
 - Ring 3 code could just jump into Ring 0 *conforming segments* and keep running!
- Non-conforming segments 😜 behave how you'd expect from a security perspective, and will throw a general protection fault if someone from a lower privilege level tries to execute them
 - Be a non-conformist! 😜



What's the Point of Conforming vs. Non-Conforming Segments? 2

- You can imagine in the original Intel notion of "OS services" that some shared code might exist in Ring 2, in a conforming segment, and the OS would want to allow Ring 3 code to just briefly jump into it and do something not *particularly* security-critical

Rings on Intel

**Operating
System
Kernel**

**Operating
System
Services**

Applications



Rings on Intel

**Operating
System
Kernel**

**Operating
System
Services**

Applications



Rings on Intel

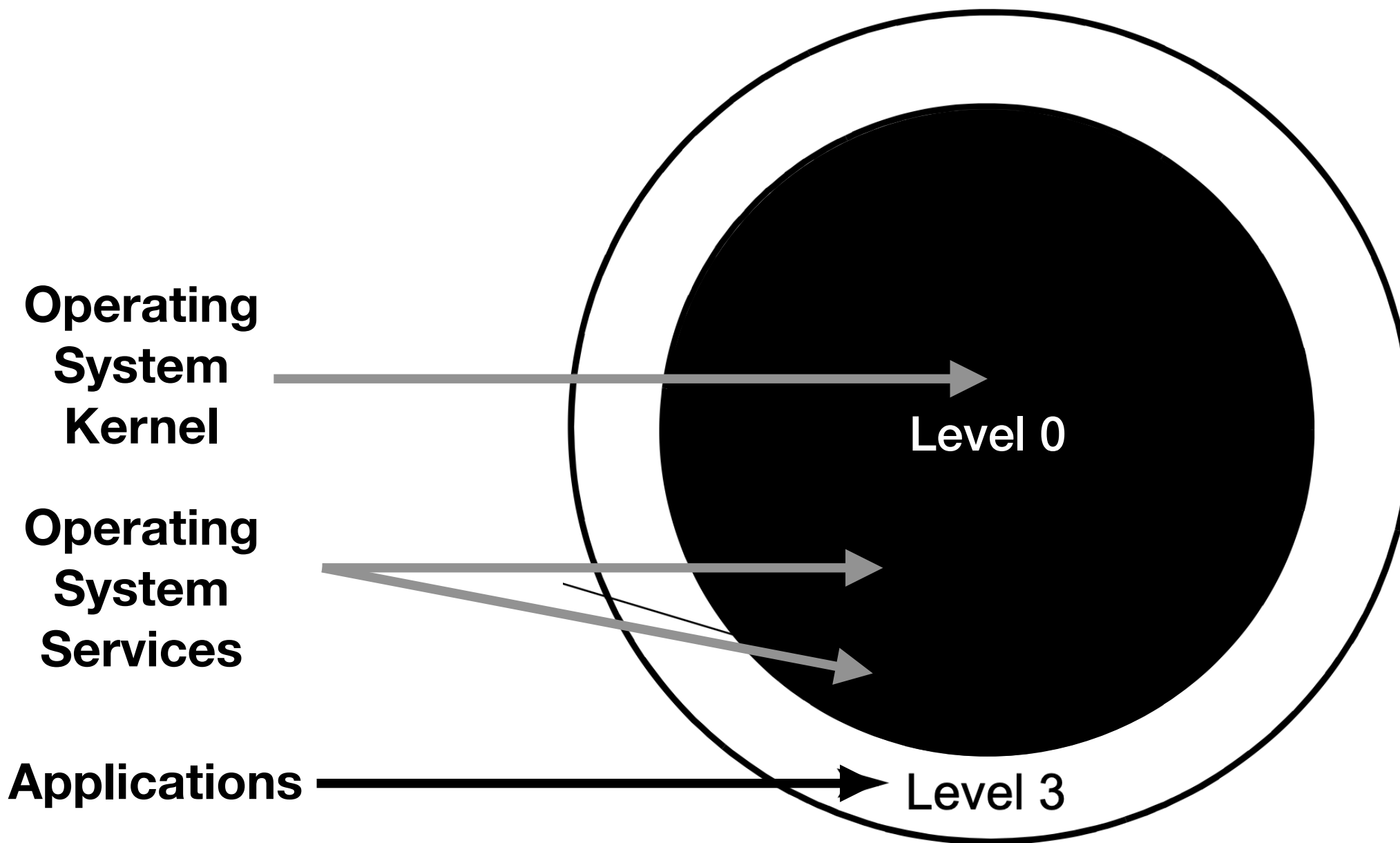


Table 3-1. Code- and Data-Segment Types

Type Field					Descriptor Type	Description
Decimal	11	10 E	9 W	8 A		
0	0	0	0	0	Data	Read-Only
1	0	0	0	1	Data	Read-Only, accessed
2	0	0	1	0	Data	Read/Write
3	0	0	1	1	Data	Read/Write, accessed
4	0	1	0	0	Data	Read-Only, expand-down
5	0	1	0	1	Data	Read-Only, expand-down, accessed
6	0	1	1	0	Data	Read/Write, expand-down
7	0	1	1	1	Data	Read/Write, expand-down, accessed
		C	R	A		
8	1	Code segments have always been defined as non-writable! This "W^X" technology had to be reinvented for paging later, since everyone was using paging rather than segmentation to protect memory 😞			Code	Execute-Only
9	1				Code	Execute-Only, accessed
10	1				Code	Execute/Read
11	1				Code	Execute/Read, accessed
12	1				Code	Execute-Only, conforming
13	1				Code	Execute-Only, conforming, accessed
14	1				Code	Execute/Read, conforming
15	1				Code	Execute/Read, conforming, accessed

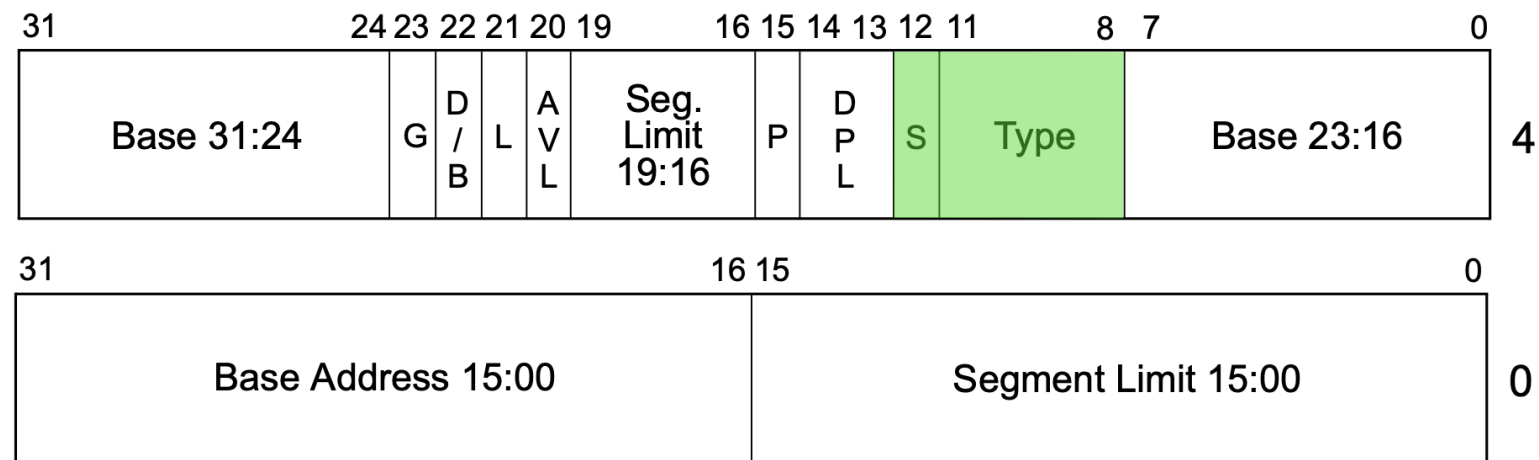
(non-conforming 😜)

Table 3-2. System-Segment and Gate-Descriptor Types

Type Field					Description	
Decimal	11	10	9	8	32-Bit Mode	IA-32e Mode
0	0	0	0	0	Reserved	Reserved
1	0	0	0	1	16-bit TSS (Available)	Reserved
2	0	0	1	0	LDT	LDT
3	0	0	1	1	16-bit TSS (Busy)	Reserved
4	0	1	0	0	16-bit Call Gate	Reserved
5	0	1	0	1	Task Gate	Reserved
6	0	1	1	0	16-bit Interrupt Gate	Reserved
7	0	1	1	1	16-bit Trap Gate	Reserved
8	1	0	0	0	Reserved	Reserved
9	1	0	0	1	32-bit TSS (Available)	64-bit TSS (Available)
10	1	0	1	0	Reserved	Reserved
11	1	0	1	1	32-bit TSS (Busy)	64-bit TSS (Busy)
12	1	1	0	0	32-bit Call Gate	64-bit Call Gate
13	1	1	0	1	Reserved	Reserved
14	1	1	1	0	32-bit Interrupt Gate	64-bit Interrupt Gate
15	1	1	1	1	32-bit Trap Gate	64-bit Trap Gate

"system descriptors in IA-32e mode are 16 bytes instead of 8 bytes" (Vol 3, Sect 3.5)

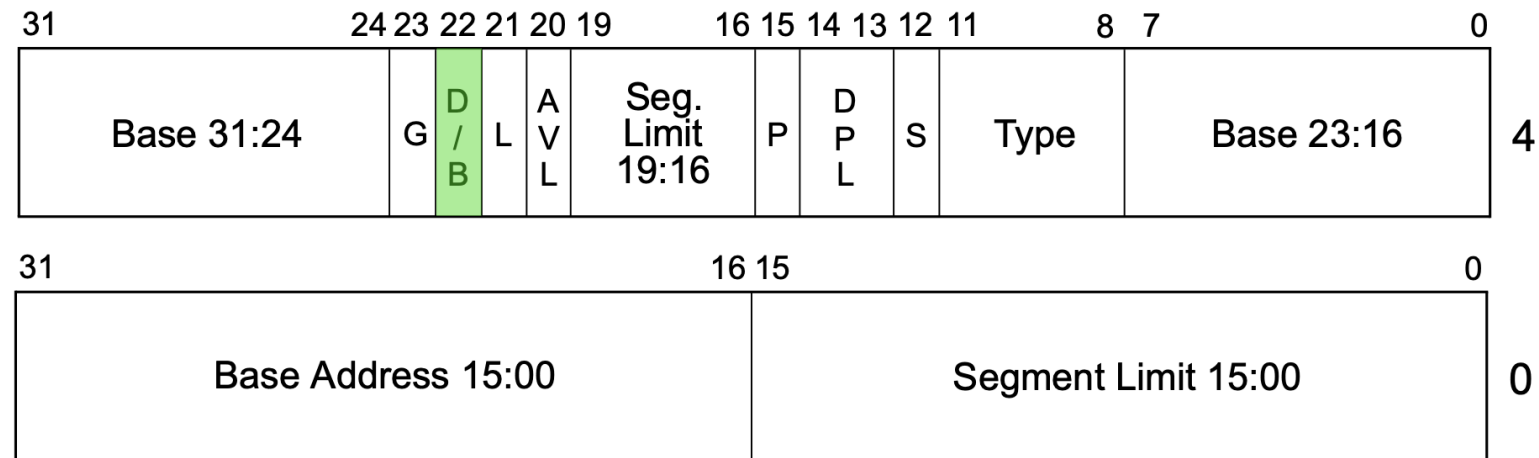
System Flag + Type Flags



- FWIW Windows combines the S and Type fields in its KGDTENTRY64 struct, to treat it like a 5-bit field of 32 types, instead of two 4-bit fields of 16 bytes

```
kd> dt -r nt!_KGDTENTRY64
+0x000 LimitLow          : Uint2B
+0x002 BaseLow           : Uint2B
+0x004 Bytes             : <anonymous-tag>
    +0x000 BaseMiddle     : UChar
    +0x001 Flags1         : UChar
    +0x002 Flags2         : UChar
    +0x003 BaseHigh       : UChar
+0x004 Bits              : <anonymous-tag>
    +0x000 BaseMiddle     : Pos 0, 8 Bits
    +0x000 Type           : Pos 8, 5 Bits
    +0x000 Dpl            : Pos 13, 2 Bits
```

D/B Flag



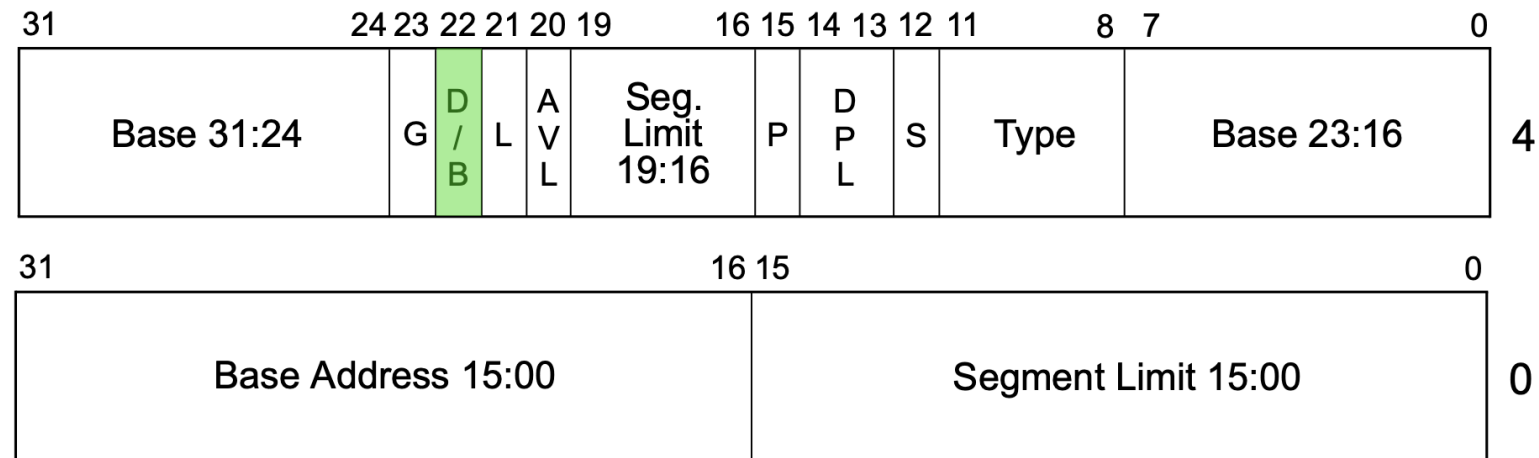
- D/B Flag: *Used for completely different stuff based on the descriptor type*
- Code Segment:
 - "D" (Default Opcode Size) flag
 - ***This*** is what actually controls whether an overloaded opcode is interpreted as dealing with 16 or 32 bit register/memory sizes

AND—Logical AND

Opcode	Instruction	Op/ En	64-bit Mode	Compat/ Leg Mode	Description
24 <i>ib</i>	AND AL, <i>imm8</i>	I	Valid	Valid	AL AND <i>imm8</i> .
25 <i>iw</i>	AND AX, <i>imm16</i>	I	Valid	Valid	AX AND <i>imm16</i> .
25 <i>id</i>	AND EAX, <i>imm32</i>	I	Valid	Valid	EAX AND <i>imm32</i> .
REX.W + 25 <i>id</i>	AND RAX, <i>imm32</i>	I	Valid	N.E.	RAX AND <i>imm32</i> sign-extended to 64-bits.

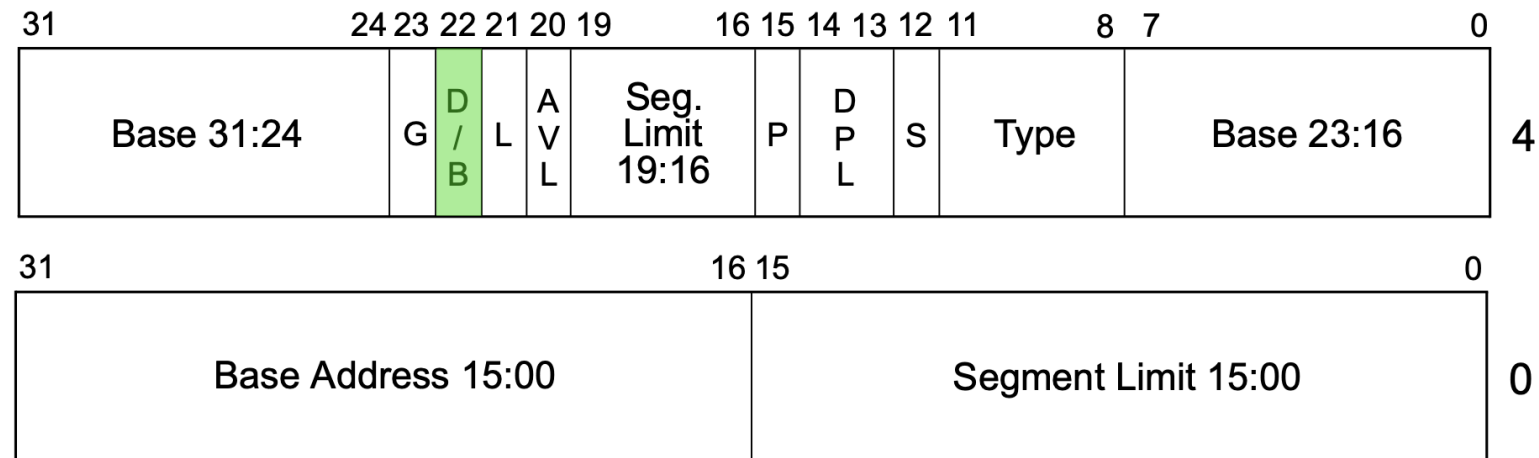
- The processor fetched a 0x25 in the instruction stream, how would it know whether it should be followed by 2 bytes (*imm16*) or 4 bytes (*imm32*)?
- The answer turns out to be: if the D flag == 0 it's 16 bits, and if D == 1 it's 32 bits!
- "The instruction prefix 66H can be used to select an operand size other than the default"

D/B Flag



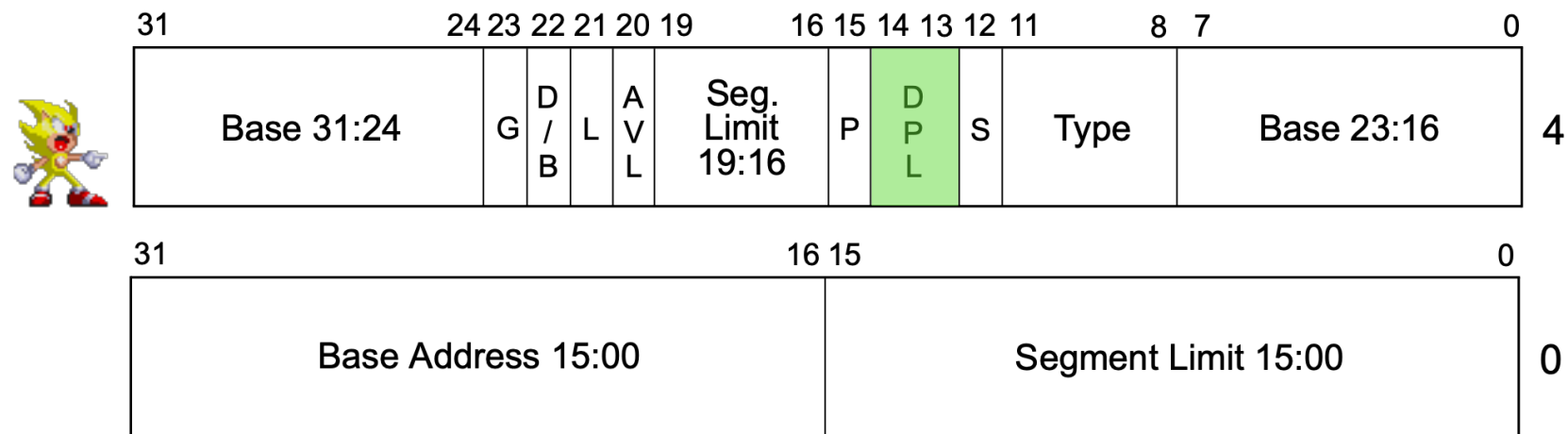
- D/B Flag: *Used for completely different stuff based on the descriptor type*
- Stack Segment (data segment pointed to by the SS register):
 - B (Big) Flag
 - Whether implicit stack pointer usage (e.g. push/pop/call) moves stack pointer by 16 bits (if B is 0), or 32 bits (if B is 1)

D/B Flag



- D/B Flag: *Used for completely different stuff based on the descriptor type*
- Expand-Down Data Segment:
 - B Flag
 - 0 = upper bound of 0xFFFF, 1 = upper bound of 0xFFFFFFFF
- No one really uses expand down segments...

Descriptor Privilege Level Flag



- DPL Flags (2 bits): Privilege Ring of the segment for access controls!

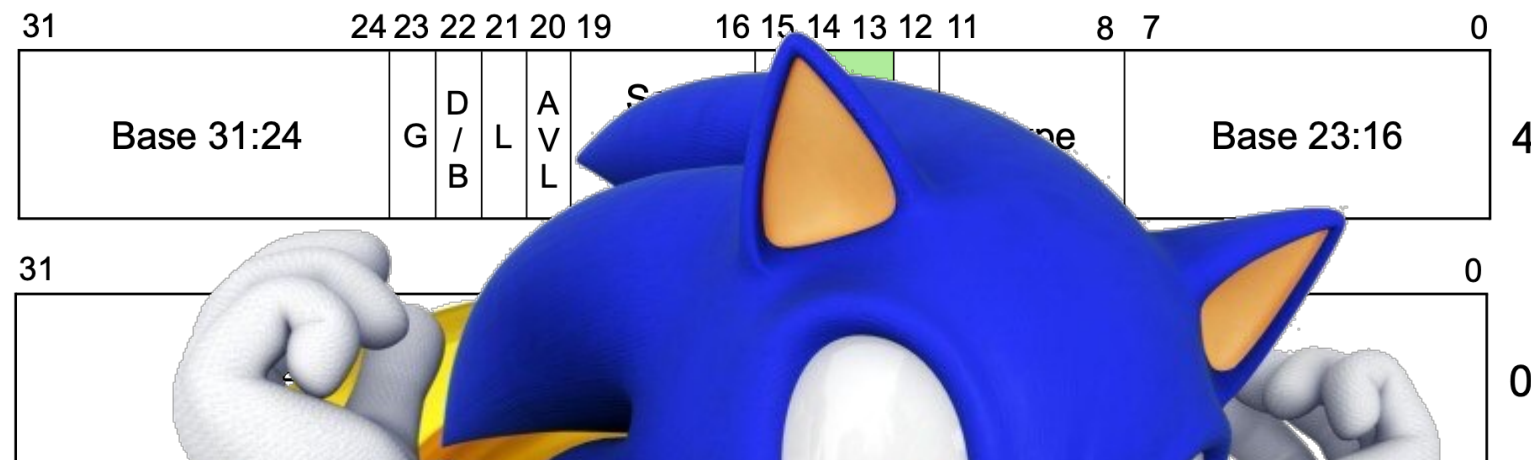


- If this is a non-conforming segment descriptor && DPL == 0, only ring 0 code can execute from within this segment



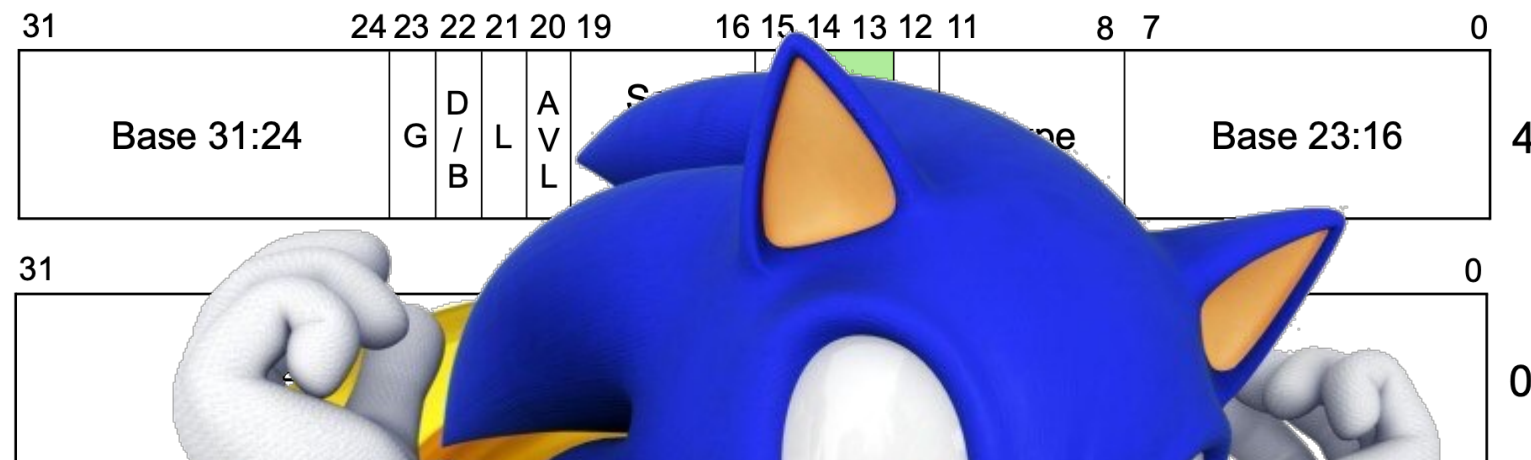
- If this is a data segment selector && DPL == 0, only ring 0 code can read/write data from/to this segment

Descriptor Privilege Level Flag



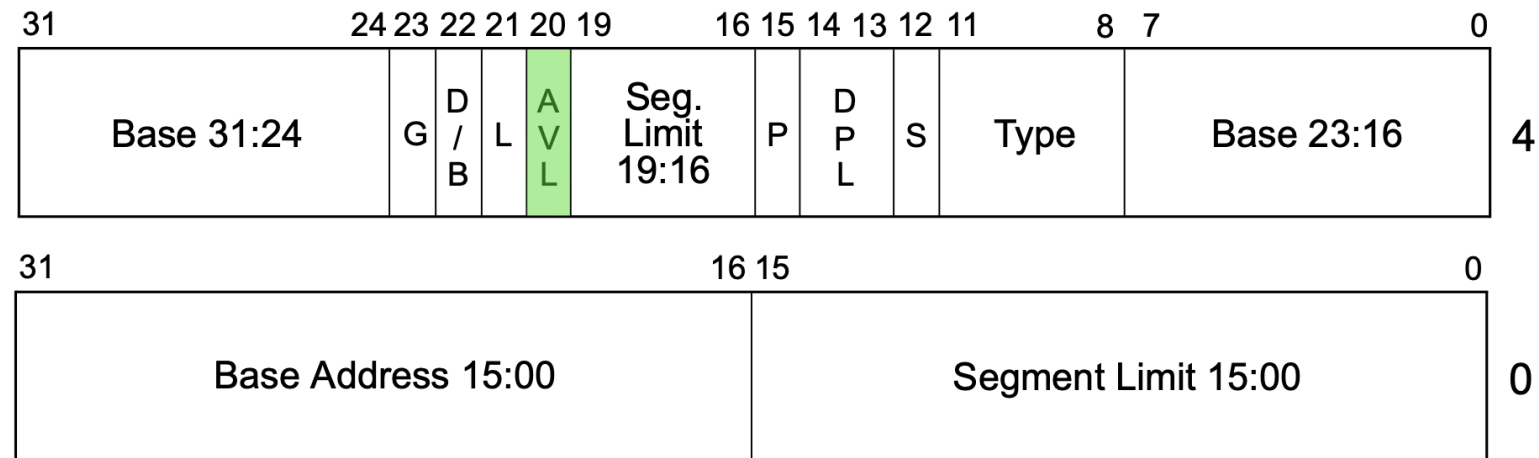
- DPL Flag: The DPL flag is a 2-bit field in the segment descriptor that indicates the privilege level of the segment. It is used to control access to the segment. If the DPL flag is 0, the segment is accessible to all code. If the DPL flag is 1, the segment is accessible only to code with a privilege level of 0 (kernel code).
- If this is a non-code segment & DPL == 0, only code with privilege level 0 can execute from within this segment
- If this is a code segment & DPL == 0, only code with privilege level 0 can write data from/to this segment

Descriptor Privilege Level Flag



- DPL Flag: The DPL flag is the privilege level of the segment. It is 0 for kernel code and 3 for user code.
- If this is a non-coding segment & DPL == 0, only code can execute from this segment
- If this is a segment && DPL == 0, only code can write data from/to this segment

Available Flag



- Available Flag: No specific usage defined. Available for OSes to use it or not use it as they see fit

Code-Segment Descriptor

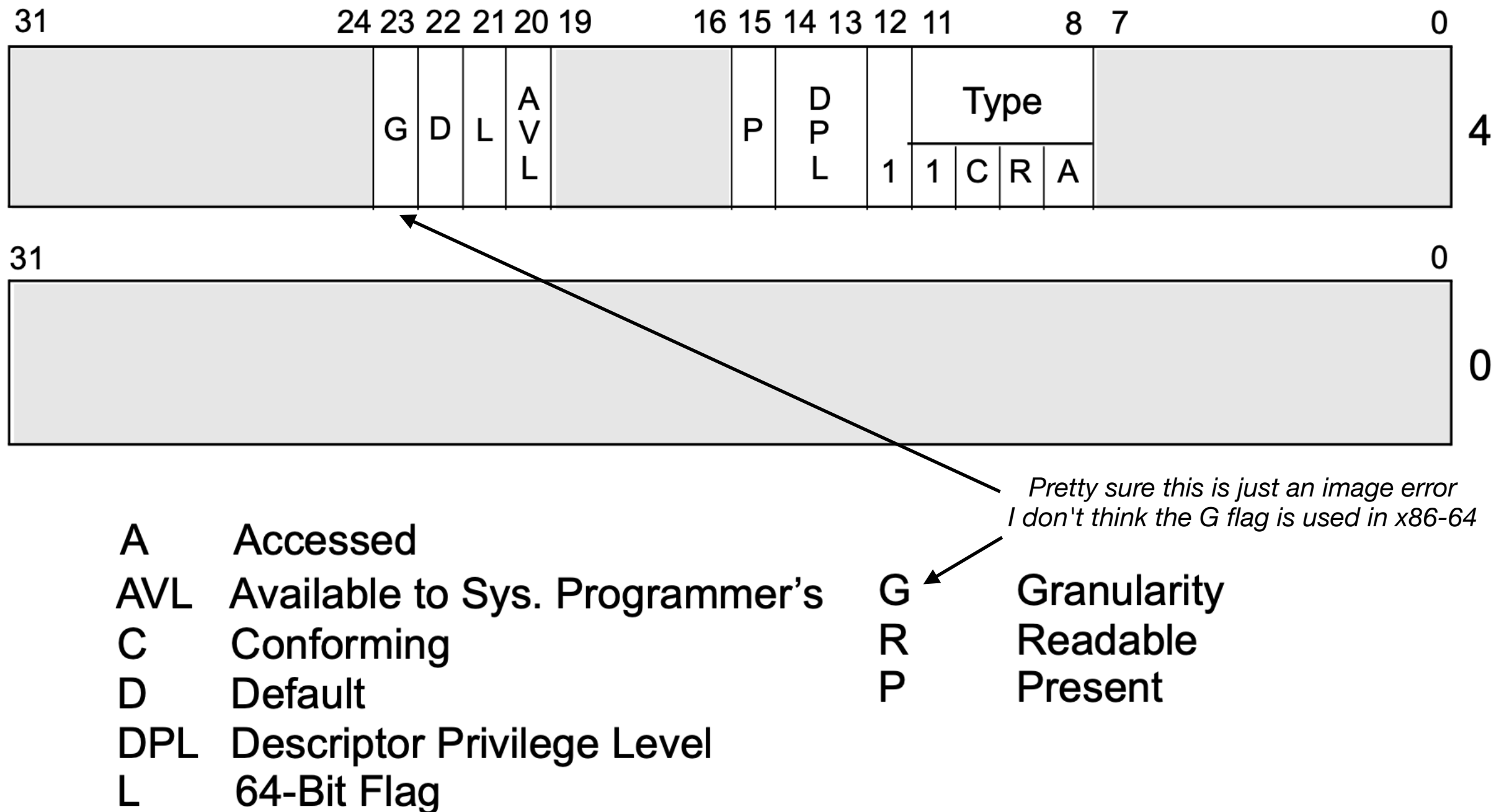
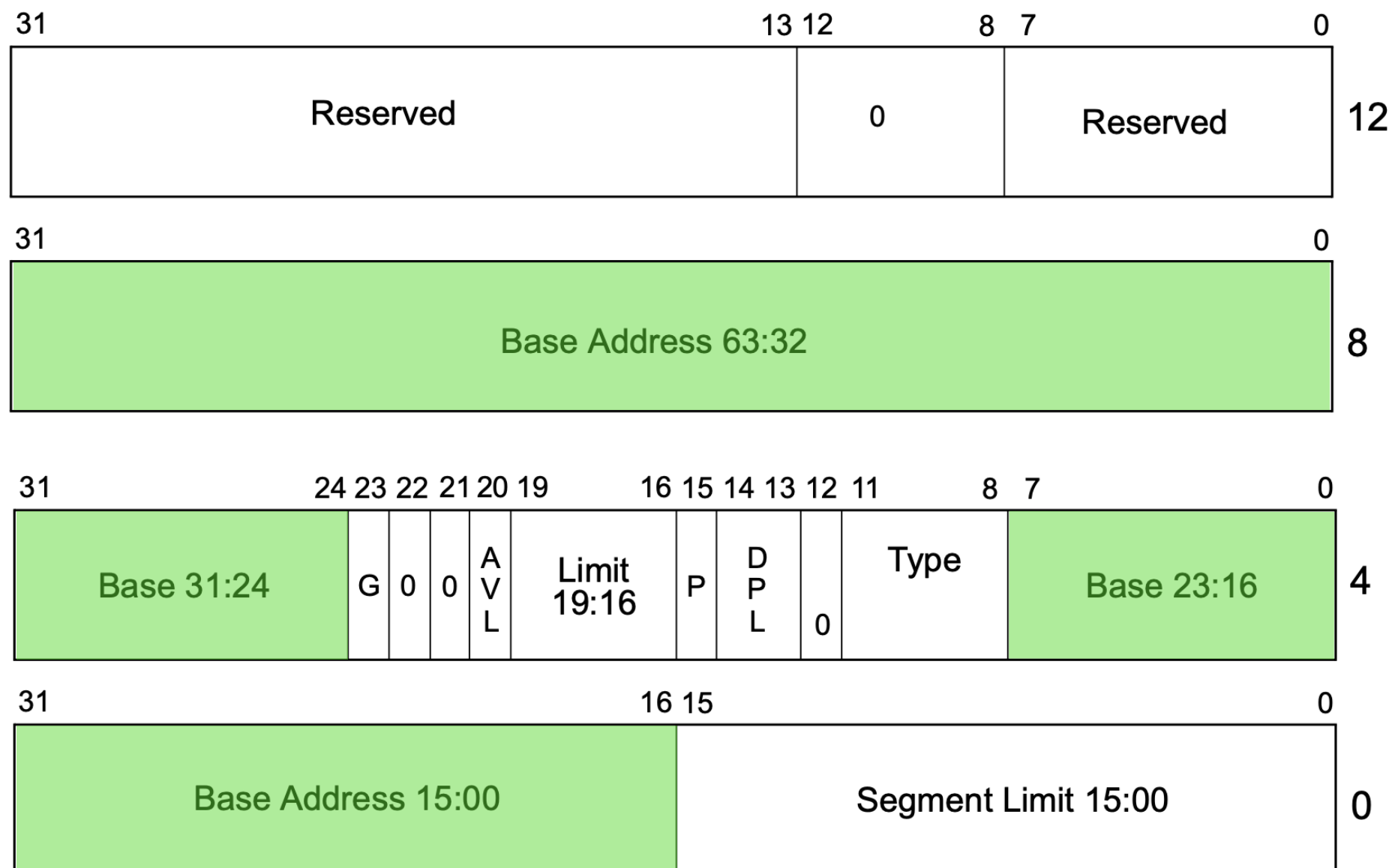


Figure 5-2. Descriptor Fields with Flags used in IA-32e Mode

TSS (or LDT) Descriptor



AVL	Available for use by system software
B	Busy flag
BASE	Segment Base Address
DPL	Descriptor Privilege Level
G	Granularity
LIMIT	Segment Limit
P	Segment Present
TYPE	Segment Type

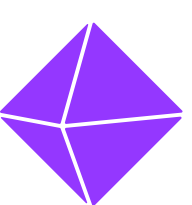
Figure 7-4. Format of TSS and LDT Descriptors in 64-bit Mode

Table 3-2. System-Segment and Gate-Descriptor Types

Type Field					Description	
Decimal	11	10	9	8	32-Bit Mode	IA-32e Mode
0	0	0	0	0	Reserved	Reserved
1	0	0	0	1	16-bit TSS (Available)	Reserved
2	0	0	1	0	LDT	LDT 
3	0	0	1	1	16-bit TSS (Busy)	Reserved
4	0	1	0	0	16-bit Call Gate	Reserved
5	0	1	0	1	Task Gate	Reserved
6	0	1	1	0	16-bit Interrupt Gate	Reserved
7	0	1	1	1	16-bit Trap Gate	Reserved
8	1	0	0	0	Reserved	Reserved
9	1	0	0	1	32-bit TSS (Available)	64-bit TSS (Available)
10	1	0	1	0	Reserved	Reserved
11	1	0	1	1	32-bit TSS (Busy)	64-bit TSS (Busy)
12	1	1	0	0	32-bit Call Gate	64-bit Call Gate
13	1	1	0	1	Reserved	Reserved
14	1	1	1	0	32-bit Interrupt Gate	64-bit Interrupt Gate
15	1	1	1	1	32-bit Trap Gate	64-bit Trap Gate

Lab: Go look at the GDT

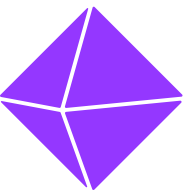
- Break into the debuggee from the debugger
- Scroll all the way down your register window to find the "gdtr" and "gdtl" registers. "gdtl" is just a WinDbg-ism for the bottom 16 bits of GDTR which specify the size (in bytes) of the overall GDT
- Enter "dq gdtr L10" to see the first 16 entries in the GDT as raw QWORDS
- Enter "dt -r (nt!_KGDTENTRY64)gdtr+(0x8*2)" to view entry index 2 (and change 2 to view other indices)

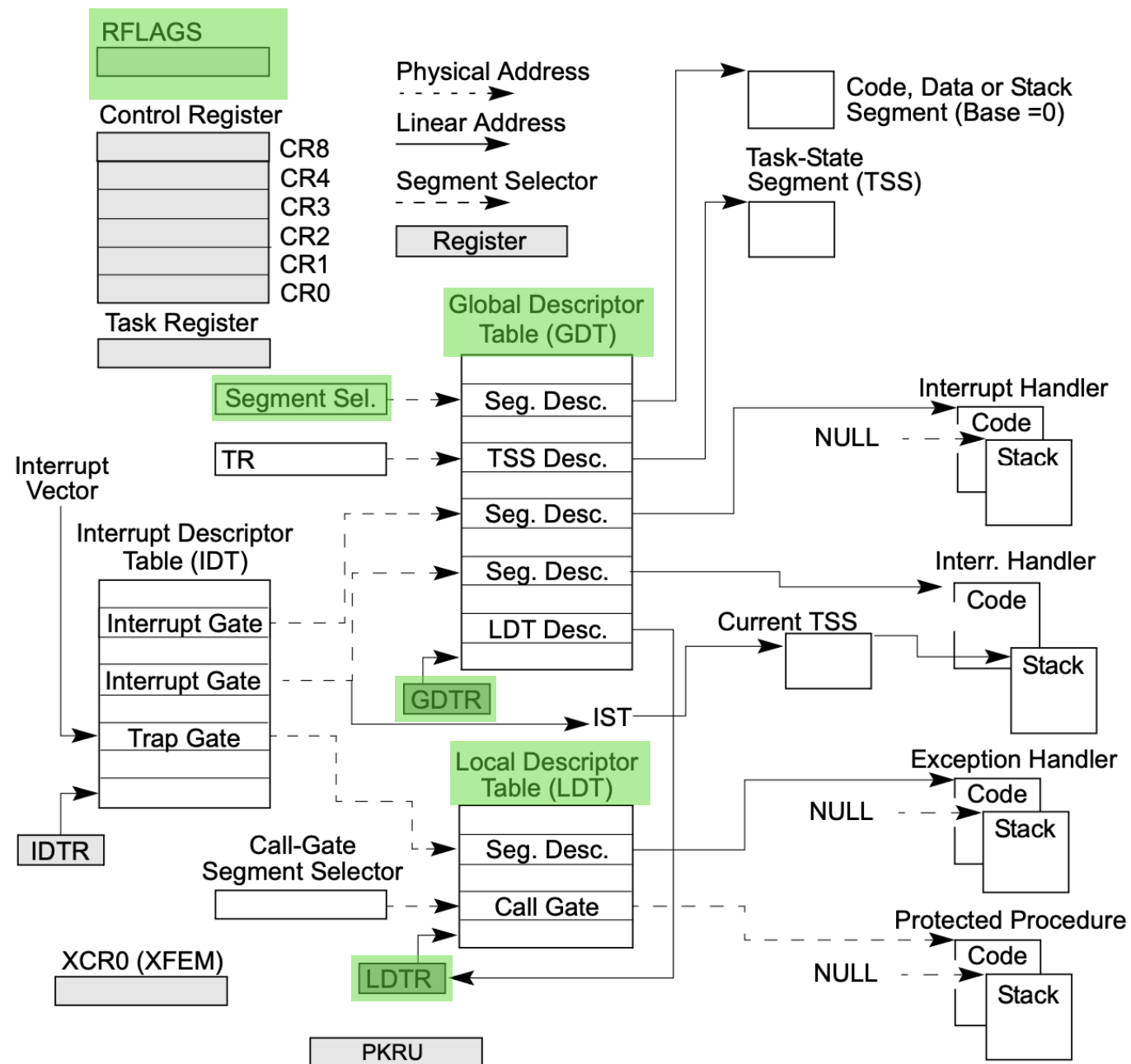


```
kd> dt -r nt!_KGDTENTRY64
+0x000 LimitLow      : Uint2B
+0x002 BaseLow       : Uint2B
+0x004 Bytes         : <anonymous-tag>
    +0x000 BaseMiddle : UChar
    +0x001 Flags1     : UChar
    +0x002 Flags2     : UChar
    +0x003 BaseHigh   : UChar
+0x004 Bits          : <anonymous-tag>
    +0x000 BaseMiddle : Pos 0, 8 Bits
    +0x000 Type       : Pos 8, 5 Bits
    +0x000 Dpl        : Pos 13, 2 Bits
```

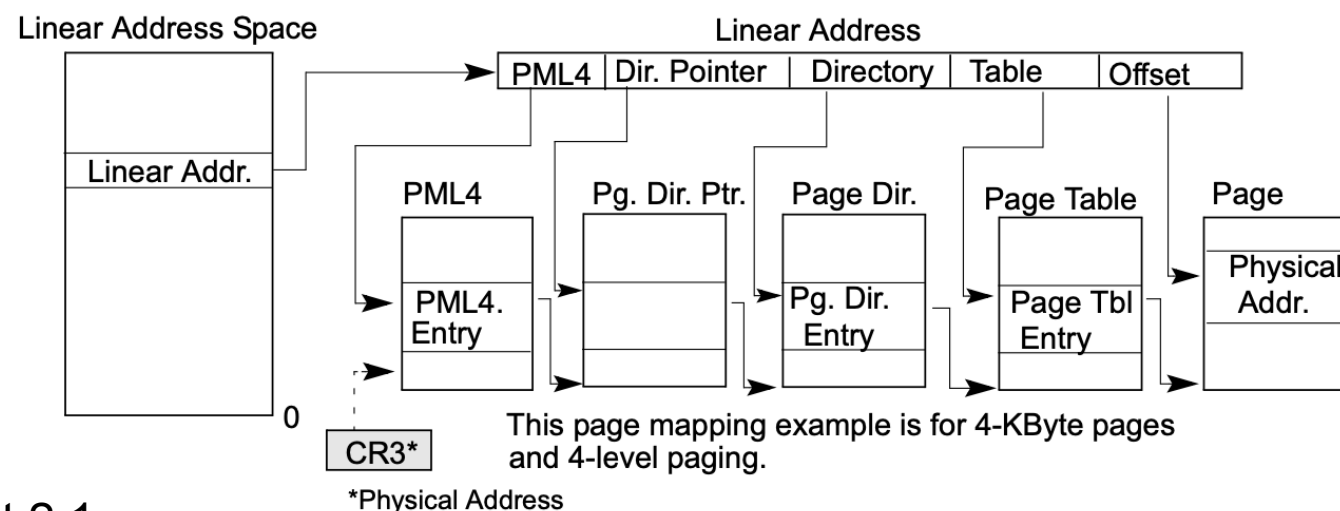
Lab: SwishDbg

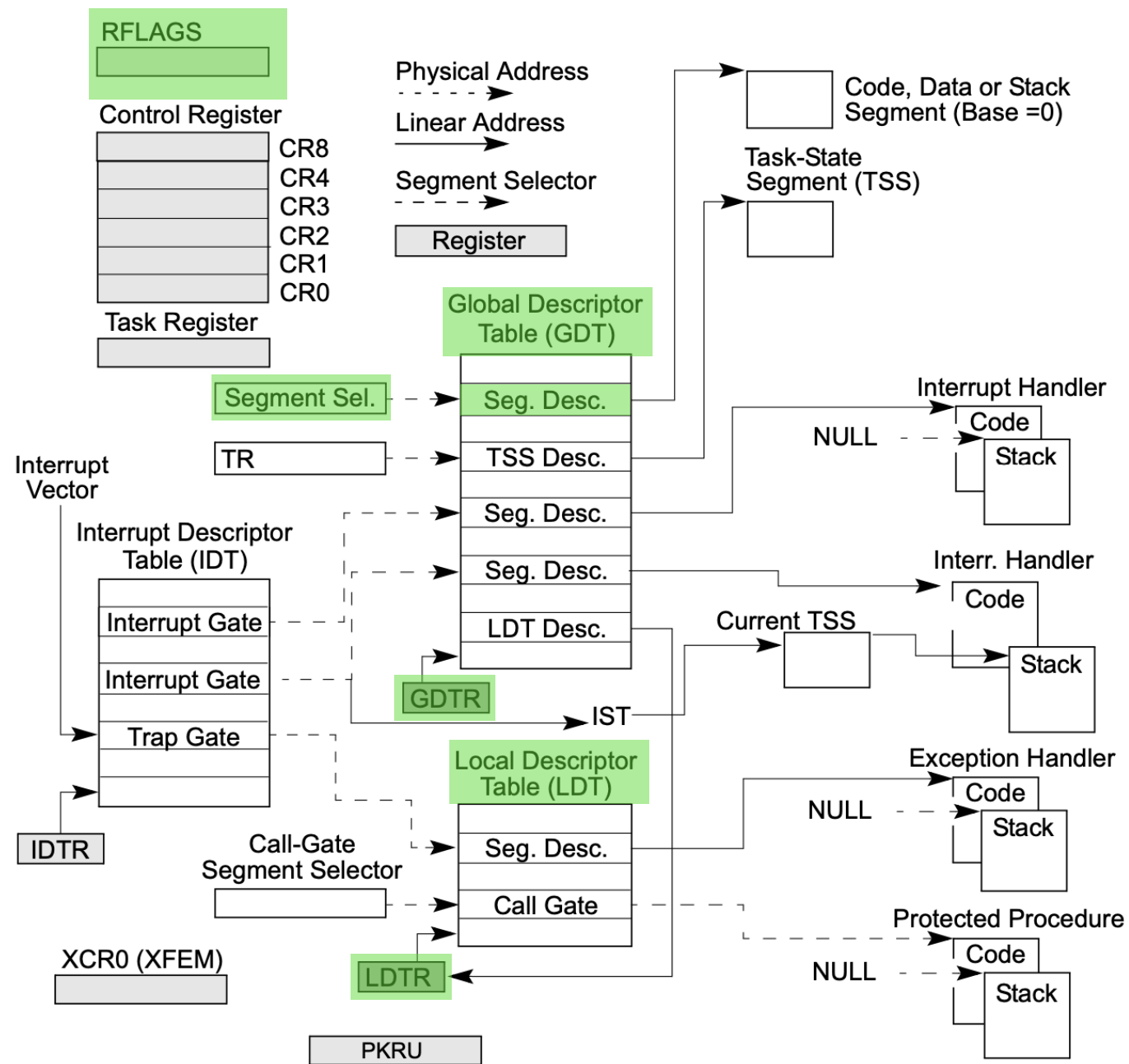
- Basic usage @ <https://github.com/comaeio/SwishDbgExt>
- !load C:\FullPathTo\SwishDbgExt.dll
 - !unload C:\FullPathTo\SwishDbgExt.dll when done
- The !ms_gdt output format was modified for this class to be more verbose



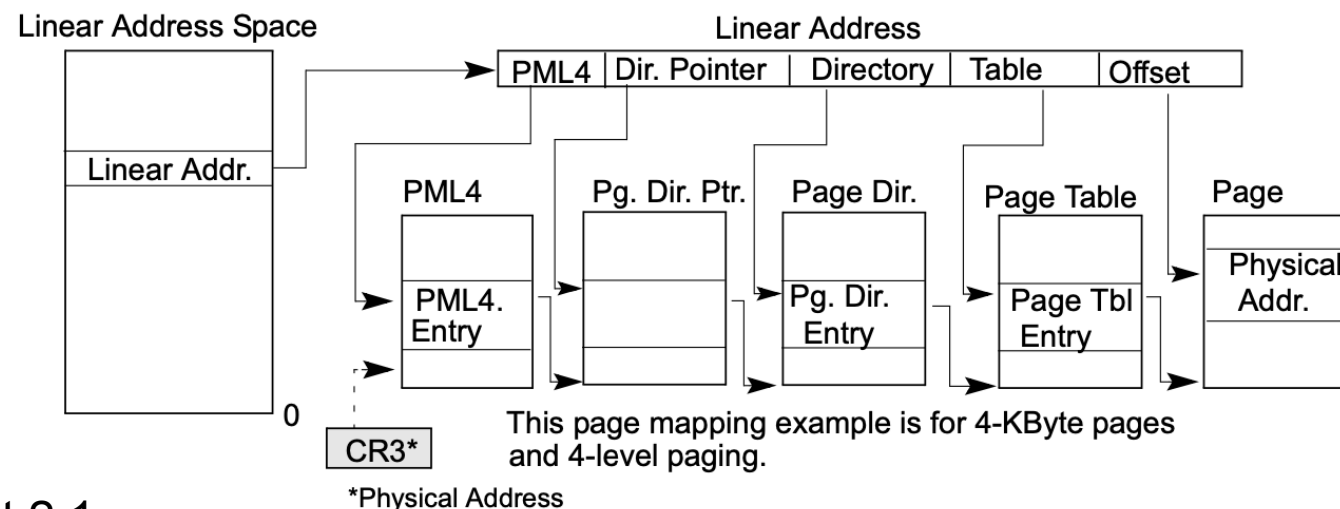


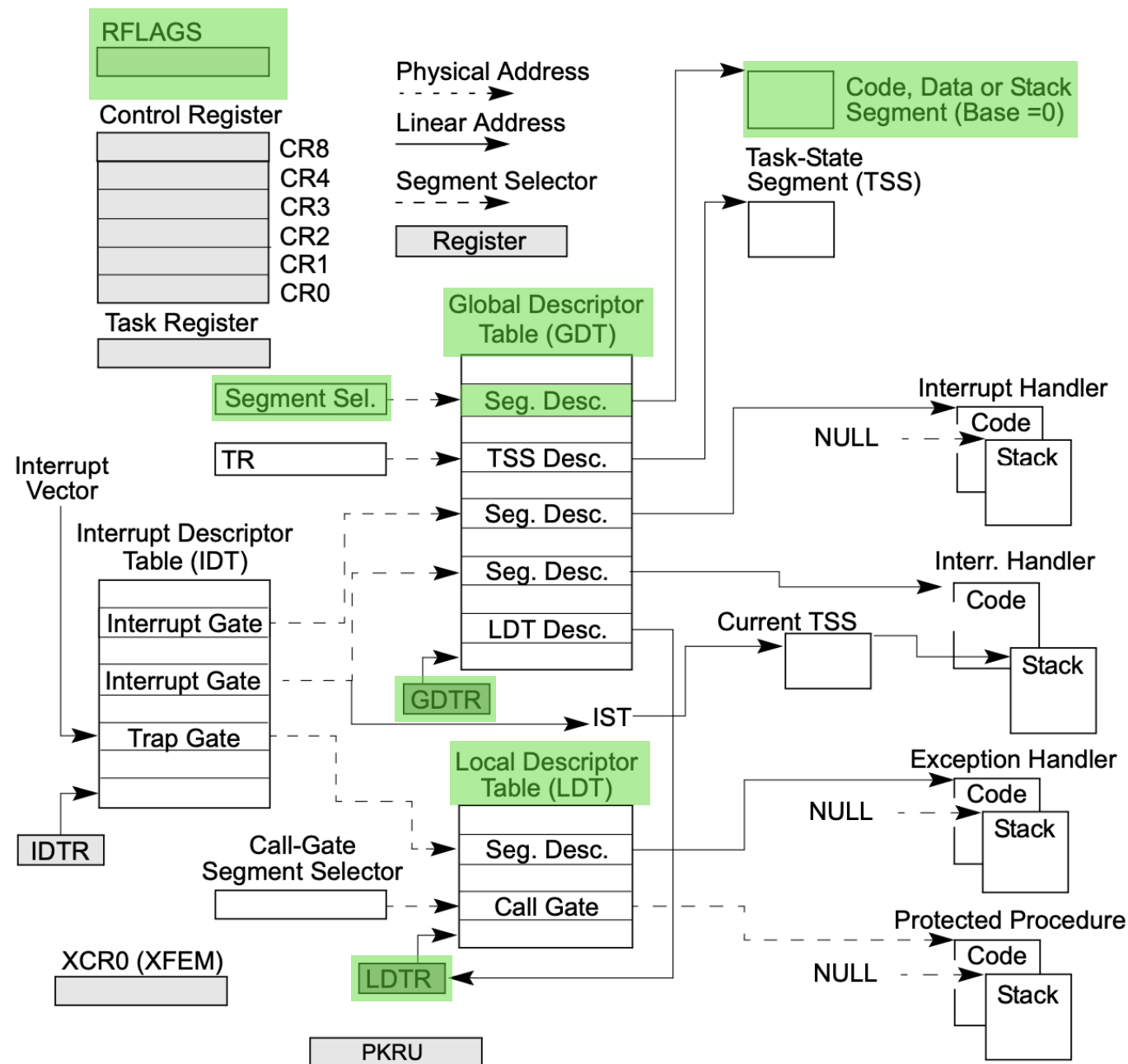
← That's what you're going to learn! :D



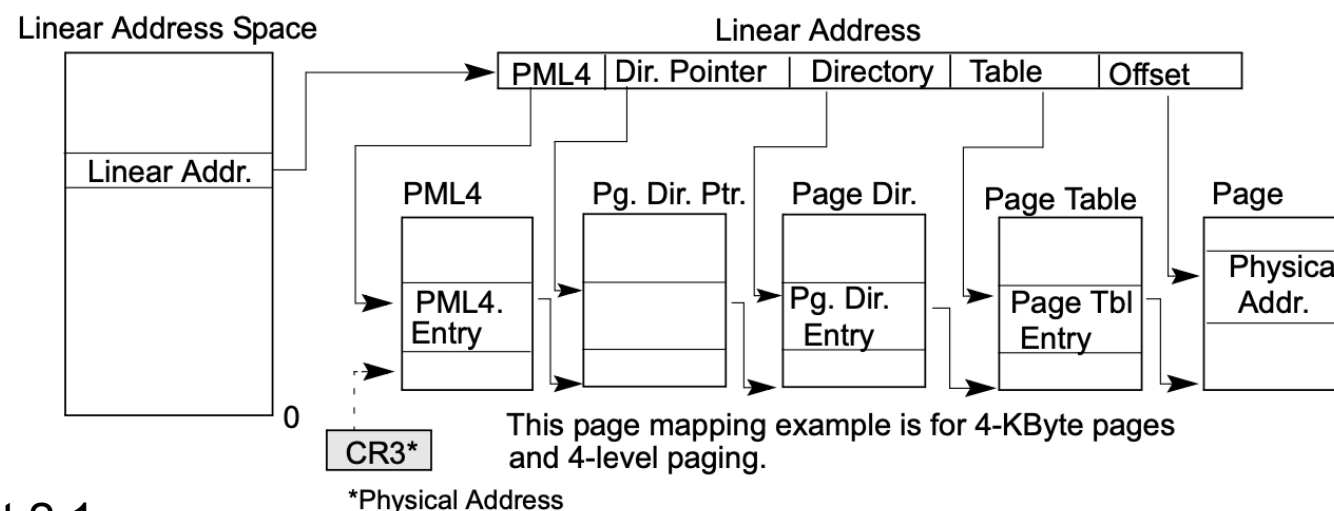


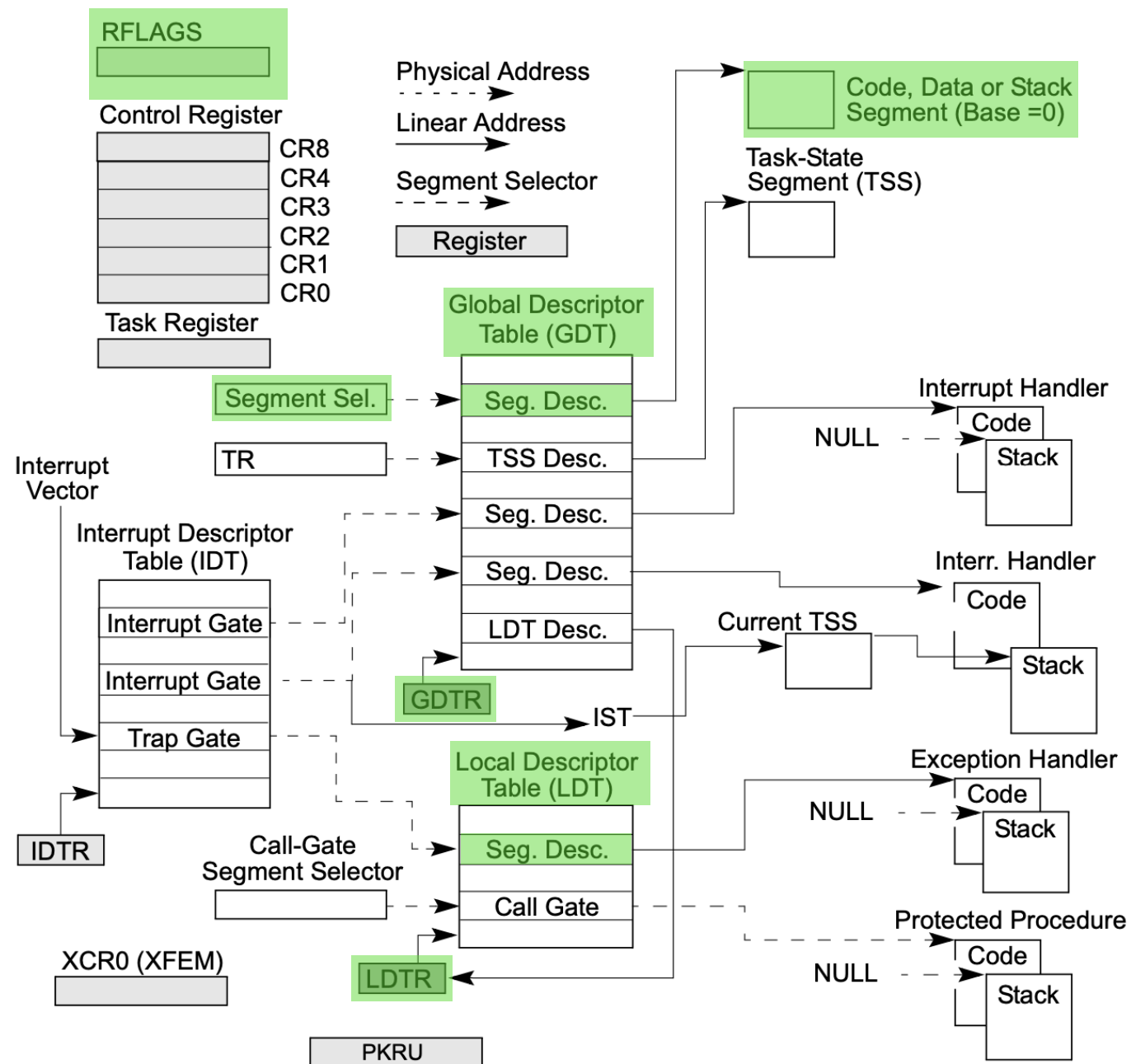
← That's what you're going to learn! :D





← That's what you're going to learn! :D





← That's what you're going to learn! :D

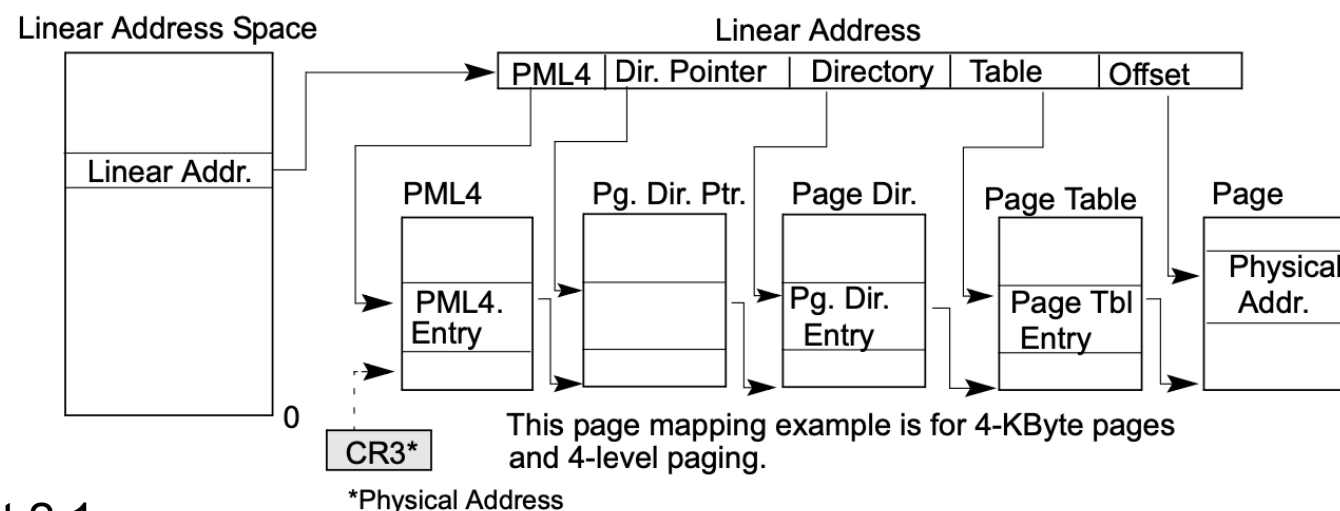
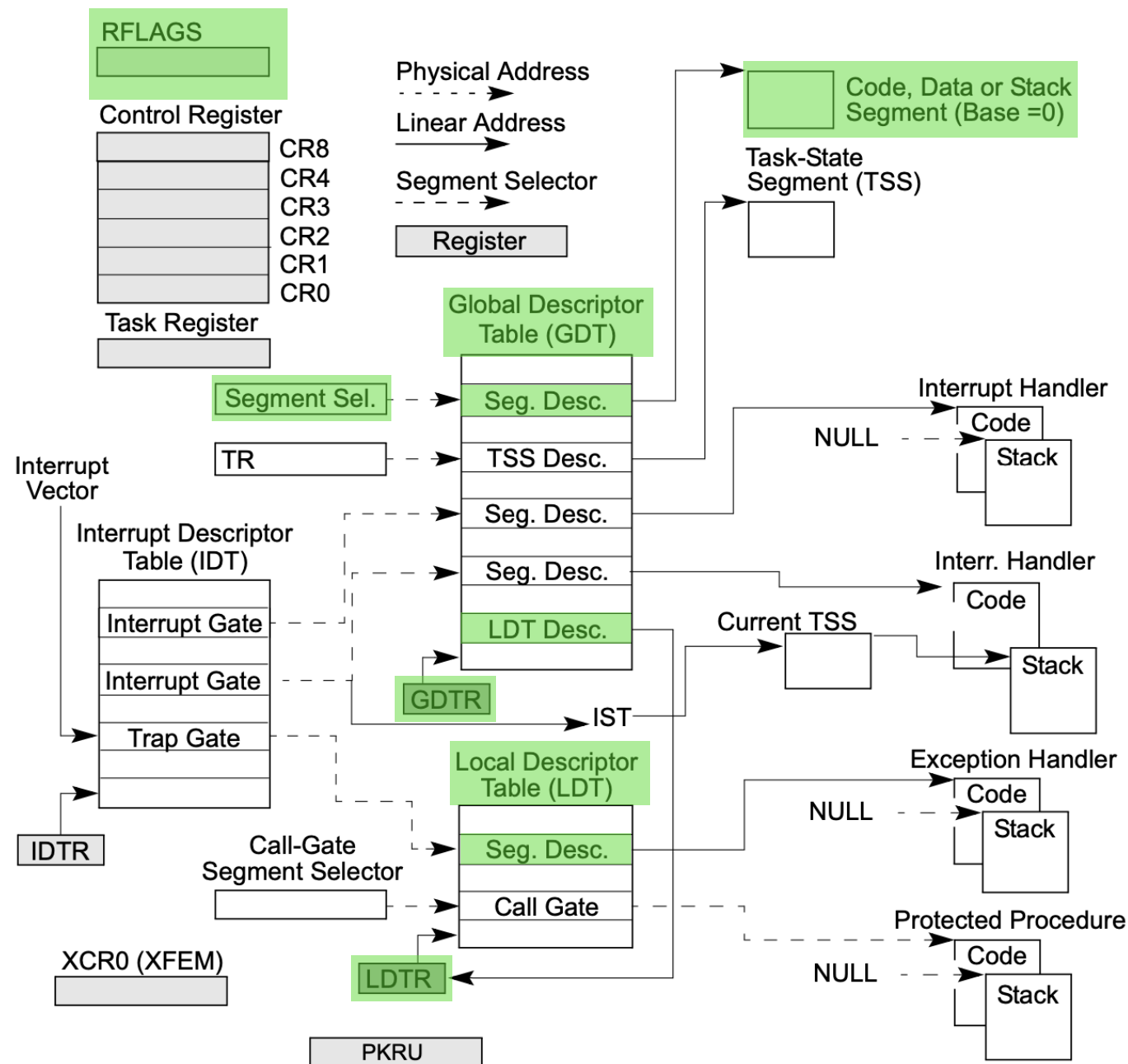


Figure 2-2. System-Level Registers and Data Structures in IA-32e Mode and 4-Level Paging



← That's what you're going to learn! :D

